FROM BEGINNER TO EXPERT

AUTOMATE EVERYTHING. ORCHESTRATE ANYWHERE. ACHIEVE MORE.

ANSIBLE

by

ABHISHEK SINGH

A COMPLETE PRACTICAL REFERENCE BOOK

A COMPLETE DEVOPS & AUTOMATION GUIDE

- AUTOMATION
- CONFIGURATION MANAGEMENT
- APPLICATION DEPLOYMENT
- INFRASTRUCTURE AS CODE
- SECURE & IDEMPOTENT
- SCALABLE & REUSABLE
- AGENTLESS ARCHITECTURE

AUTOMATE. DEPLOY. SCALE. REPEAT.

REAL-WORLD SOLUTIONS

PRODUCTION READY

ANSIBLE | AWS | AZURE | GCP | KUBERNETES | DOCKER | GITHUB | JENKINS

INFRASTRUCTURE AUTOMATION
Provision, Configure & Manage Infrastructure Automatically

CLOUD-NATIVE ARCHITECTURES
Build Resilient, Scalable & Highly Available Solutions

INFRASTRUCTURE AS CODE
Version Controlled Infrastructure with Reusability

CI/CD & DEVOPS INTEGRATION
Automate Build, Test and Deployment End-to-End

SECURITY & BEST PRACTICES
Secure by Design with Compliance & Governance

MONITORING & OBSERVABILITY
Monitor, Log and Optimize Performance in Real-Time

BEGINNER TO ADVANCED
Step-by-step learning path for all levels

HANDS-ON LABS
Practical examples and exercises

INTERVIEW QUESTIONS
Top 100 Ansible Q&A with answers

REAL-TIME SCENARIOS
Industry use cases & problem-solving

PRODUCTION FOCUSED
Best practices for real-world success

YOUR ROADMAP TO BECOMING AN

ANSIBLE EXPERT

LEARN ★ PRACTICE ★ AUTOMATE ★ DEPLOY ★ SUCCEED

★ COMPLETE 30-PAGE ROADMAP TO ACE ANY ANSIBLE INTERVIEW ★

COMPLETE ANSIBLE SYLLABUS

ANSIBLE

30 PAGES ROADMAP – FROM BEGINNER TO EXPERT

Beginner to Expert | Hands-on Learning | Real-world Projects | Interview & Certification | Best Practices

01. INTRODUCTION TO ANSIBLE
- What is Ansible?
- Why Ansible?
- Ansible vs Other Tools
- Key Features
- Architecture Overview
- Use Cases

Page 01

02. INSTALLATION & SETUP
- System Requirements
- Install Ansible (Linux)
- Install Ansible (Windows)
- Verify Installation
- Ansible Directory Structure
- Configuration Files

Page 02

03. INVENTORY MANAGEMENT
- What is Inventory?
- Inventory File Formats
- INI Inventory
- YAML Inventory
- Host Groups
- Host Variables

Page 03

04. AD-HOC COMMANDS & MODULES
- What are Ad-hoc Commands?
- Syntax
- Common Modules
- Command Module
- Shell Module
- Copy, File, Ping Modules

Page 04

05. PLAYBOOKS FUNDAMENTALS
- What are Playbooks?
- Playbook Structure
- Tasks & Plays
- Run a Playbook
- Check Mode
- Diff Mode

Page 05

06. YAML BASICS
- YAML Syntax
- Indentation
- Lists & Dictionaries
- Comments
- Best Practices

Page 06

07. VARIABLES IN ANSIBLE
- What are Variables?
- Define Variables
- Variable Precedence
- Magic Variables
- Facts vs Variables
- Register Variables

Page 07

08. FACTS & GATHERING
- What are Facts?
- Setup Module
- View Facts
- Custom Facts
- Facts Caching
- Use Cases

Page 08

09. LOOPS & ITERATIONS
- What are Loops?
- Loop with_items
- Loop with_list
- Loop with_dict
- Loop Control
- Loop Best Practices

Page 09

10. CONDITIONS & ERROR HANDLING
- When Condition
- When with Operators
- Failed_when
- Ignored_errors
- Block/Rescue/Always
- Assert Module

Page 10

11. HANDLERS & NOTIFICATIONS
- What are Handlers?
- Notify & Listen
- Trigger Handlers
- Flush Handlers
- Handler Best Practices

Page 11

12. TEMPLATES & JINJA2
- Templates Module
- Jinja2 Basics
- Variables in Templates
- Filters
- Conditionals in Jinja2
- Loops in Jinja2

Page 12

13. ROLES & REUSABILITY
- What are Roles?
- Role Structure
- Create a Role
- Role Dependencies
- Galaxy Roles
- Best Practices

Page 13

14. ANSIBLE GALAXY & COLLECTIONS
- What is Ansible Galaxy?
- Search Roles
- Install Roles
- Collections Overview
- Install Collections
- Use Collections

Page 14

15. VAULT & ENCRYPTION
- What is Ansible Vault?
- Encrypt Variables
- Encrypt Files
- Edit Encrypted Files
- Vault Passwords
- Best Practices

Page 15

16. TAGS & SELECTIVE EXECUTION
- What are Tags?
- Add Tags to Tasks
- Run with Tags
- Skip Tags
- Tag Best Practices

Page 16

17. DYNAMIC INVENTORY
- What is Dynamic Inventory?
- Scripted Inventory
- AWS Dynamic Inventory
- Plugin-based Inventory
- Benefits
- Best Practices

Page 17

18. ADVANCED PLAYBOOKS
- Includes vs Imports
- Vars Files
- Become (Privilege Escalation)
- Async & Poll
- Strategies
- Run_once

Page 18

19. CLOUD AUTOMATION WITH ANSIBLE (AWS)
- AWS Overview
- Configure AWS
- EC2 Automation
- S3 Automation
- RDS Automation
- Best Practices

Page 19

20. DOCKER AUTOMATION WITH ANSIBLE
- Docker Overview
- Install Docker
- Manage Containers
- Docker Images
- Docker Compose
- Best Practices

Page 20

21. KUBERNETES AUTOMATION WITH ANSIBLE
- Kubernetes Overview
- Install Kubernetes
- Ansible K8s Modules
- Deployments
- Services
- ConfigMaps & Secrets
- Helm Integration

Page 21

22. CI/CD INTEGRATION WITH ANSIBLE
- CI/CD Overview
- Jenkins Integration
- GitHub Actions
- GitLab Integration
- Azure DevOps
- Ansible in Pipelines

Page 22

23. REAL-TIME PROJECT DESIGN (END-TO-END)
- Project Architecture
- GitHub → Jenkins
- Build → Docker
- Push Image
- Ansible Deployment
- Kubernetes Deployment

Page 23

24. PRODUCTION TROUBLESHOOTING
- Playbook Failed
- SSH Access Denied
- Inventory Issues
- Variables Not Loading
- Service Not Restarting
- Vault Password Failure
- Debugging Tips

Page 24

25. ADVANCED CONCEPTS
- Async Tasks
- Polling
- Delegation
- Local Actions
- Serial Execution
- Rolling Updates
- Parallel Execution

Page 25

26. REAL-TIME INTERVIEW SCENARIOS
- Deploy to 500 Servers
- Restart Failed Services
- Patch Linux Servers
- Rotate SSL Certificates
- AWS Infra + Deploy
- Blue-Green Deployment
- Canary Deployment
- Disaster Recovery

Page 26

27. TOP 100 ANSIBLE INTERVIEW QUESTIONS
- Beginner (1-25)
- Intermediate (26-50)
- Advanced (51-75)
- Scenario-Based (76-100)
- Concepts & Theory
- Practical Questions
- Best Answers

Page 27

28. COMPLETE ANSIBLE INTERVIEW REVISION
- Must Know Topics
- Key Concepts
- Important Commands
- Quick Revision Notes
- Cheat Sheet
- Important Points

Page 28

29. REAL-WORLD PROJECTS
- Web Application Deployment
- Multi-tier Architecture
- Monitoring Setup
- Backup Automation
- User Provisioning
- Log Management
- End-to-End Projects

Page 29

30. BONUS & NEXT STEPS
- Certifications
- Career Paths
- Interview Tips
- Resume Tips
- Learning Resources
- Community & Support
- Keep Learning

Page 30

WHY LEARN ANSIBLE?
- Agentless & Simple
- Powerful Automation
- Huge Community
- Cloud & DevOps Ready
- High Demand in Market

KEY TOOLS & INTEGRATIONS
AWS, Azure, GCP, Docker, Kubernetes, Jenkins, GitHub Actions, GitLab, Prometheus, Grafana, Vault, Terraform, Linux, Windows, VMware

CAREER OUTCOMES
- DevOps Engineer
- Cloud Engineer
- Site Reliability Engineer (SRE)
- Platform Engineer
- Automation Engineer
- Infrastructure Engineer

WHAT YOU WILL MASTER
Automation | Configuration Management | Infrastructure as Code | Cloud Automation | CI/CD Integration | Monitoring & Management | Security & Compliance

START YOUR AUTOMATION JOURNEY TODAY!
LEARN ★ PRACTICE ★ AUTOMATE ★ DEPLOY ★ SUCCEED

BUILD SKILLS. BUILD PROJECTS. BUILD YOUR CAREER.

This 30-page roadmap is designed to take you from Beginner to Expert in Ansible.

Hands-on Learning | Real-world Projects | Interview Ready | Production Ready

BY ABHISHEK SINGH

Ansible Essentials

By Abhishek Singh

1. INTRODUCTION TO ANSIBLE

Ansible is an open-source IT automation tool used for configuration management, application deployment, and infrastructure automation. It helps you manage systems simply and efficiently.

TOPICS COVERED

- What is Ansible?
- Why Ansible?
- Configuration Management Overview
- Infrastructure Automation
- Agentless Architecture
- Push vs Pull Architecture
- Benefits of Ansible
- DevOps Use Cases
- Ansible in Real Projects
- Interview Questions

KEY CONCEPTS AT A GLANCE

1. What is Ansible?

Ansible is an open-source automation tool that helps in configuration management, application deployment, orchestration, and continuous delivery.

2. Why Ansible?

- Easy to learn & use
- Agentless architecture
- Uses simple YAML syntax
- Huge community support
- Great for DevOps & Cloud

3. Configuration Management Overview

Configuration management ensures systems are in the desired state by automating software installation, updates, user management, and more.

4. Infrastructure Automation

Ansible automates provisioning, configuration, deployment, scaling, and management of infrastructure across on-prem and cloud environments.

5. Agentless Architecture

- No agents required on remote systems
- Uses SSH for Linux and WinRM for Windows
- Simplifies deployment and management

6. Push vs Pull Architecture

Push: Ansible (Control Node) pushes configurations to managed nodes.
Pull: Other tools (like Puppet, Chef) require agents to pull configurations.

7. Benefits of Ansible

- Simple & human-readable
- Faster automation
- Secure (SSH based)
- Idempotent tasks
- Cross-platform support

8. DevOps Use Cases

- Continuous Integration
- Continuous Deployment
- Infrastructure as Code
- Application Deployment
- Cloud Automation

9. Ansible in Real Projects

- Web server provisioning
- Database setup and configuration
- Application deployment
- Cloud infrastructure management

10. Interview Questions

- What is Ansible?
- How does Ansible work?
- Ansible vs Puppet vs Chef?
- What is inventory in Ansible?
- How do playbooks work?

IMPORTANT FAQs

1. What is Ansible?

Ansible is an open-source automation tool used for configuration management, application deployment, orchestration, and infrastructure automation. It is agentless and uses simple YAML-based playbooks.

2. Why is Ansible preferred over Puppet and Chef?

- **Agentless:** No software to install on managed nodes.
- **Simple:** Easy YAML language.
- **Fast:** Lightweight & efficient.
- **Flexible:** Works across multiple platforms and environments.

3. What problems does Ansible solve?

- Reduces manual repetitive tasks
- Ensures consistency across systems
- Speeds up application deployment
- Simplifies infrastructure management
- Improves reliability and scalability

DID YOU KNOW?

Ansible was created by Michael DeHaan and is now part of Red Hat.

IMPORTANT NOTE

- Ansible works over SSH (Linux) and WinRM (Windows).
- No agents means lower overhead and easier maintenance.

REMEMBER

Automate Everything, Manage Anything!

ONE LINE SUMMARY

Ansible is a simple, agentless automation tool that helps you manage and automate your infrastructure efficiently.

ANSIBLE

ANSIBLE ARCHITECTURE

By Abhishek Singh

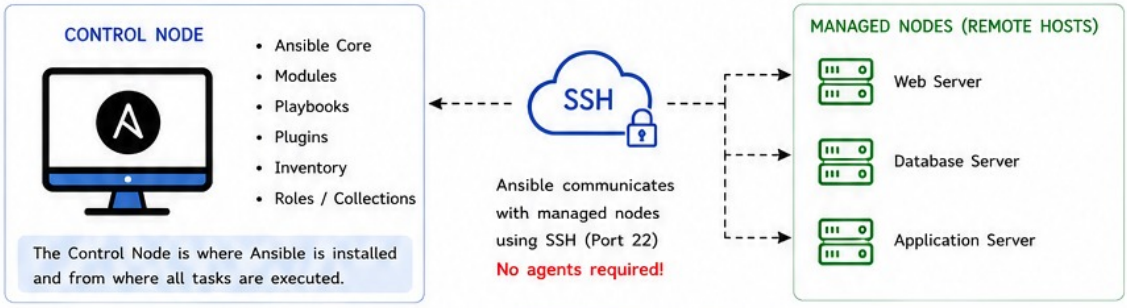
2. ANSIBLE ARCHITECTURE

Ansible follows an agentless architecture where the Control Node manages and communicates with Managed Nodes over SSH to automate tasks.

TOPICS COVERED

- Control Node
- Managed Nodes
- Inventory
- SSH Communication
- Modules
- Playbooks
- Roles
- Collections
- Interview Questions

ANSIBLE ARCHITECTURE OVERVIEW



KEY COMPONENTS AT A GLANCE

1. Control Node
- Machine where Ansible is installed
- Executes playbooks
- Connects to managed nodes via SSH

2. Managed Nodes
- Remote servers/devices managed by Ansible
- No agent installed
- Any OS (Linux/Unix/Windows)

3. Inventory
- List of managed hosts and groups
- Can be static (INI) or dynamic (scripts, plugins, cloud, etc.)

4. SSH Communication
- Ansible uses SSH to connect to managed nodes
- Recommended to use key-based authentication

5. Modules
- Reusable units of code to perform specific tasks
- Hundreds of built-in modules

6. Playbooks
- YAML files that define tasks, variables, handlers and more
- Executed on managed nodes

7. Roles
- Organize playbooks, tasks, files, templates, handlers in a structured way
- Promotes reusability

8. Collections
- Distributions of content (roles, modules, plugins)
- Managed and shared via Ansible Galaxy

9. Interview Questions
- Explain Ansible Architecture.
- What is Control Node?
- How does Ansible communicate with remote servers?

IMPORTANT CONCEPTS

1. Explain Ansible Architecture.

Ansible uses an agentless architecture. The Control Node connects to Managed Nodes over SSH and executes modules defined in playbooks to automate tasks.

2. What is Control Node?

The Control Node is the machine where Ansible is installed. It acts as the brain of Ansible and manages all automation activities on remote nodes.

3. How does Ansible communicate with remote servers?
- Ansible communicates using SSH (Port 22)
- No agents are required on remote servers
- Uses key-based authentication (recommended)
- Supports sudo/su for privilege escalation

DID YOU KNOW?

Ansible works on Linux, Windows, BSD, network devices, cloud platforms and more!

IMPORTANT NOTE
- Keep SSH access enabled on managed nodes.
- Use strong SSH keys for secure automation.
- Ansible does not require any database.

REMEMBER

Agentless. Simple. Powerful. That's Ansible!

ONE LINE SUMMARY

Ansible uses an agentless architecture where the Control Node connects to managed nodes over SSH and automates tasks using modules and playbooks.

ANSIBLE

INSTALLING ANSIBLE

By Abhishek Singh

3. INSTALLING ANSIBLE

Ansible can be installed on various platforms. It requires Python and SSH access to managed nodes. Below are the common installation methods and related information.

TOPICS COVERED

- Installation on Linux
- Installation on AWS EC2
- Installation on Ubuntu
- Installation on RHEL
- Ansible Version Verification
- Configuration File
- Hands-On Commands
- Interview Questions

INSTALLATION METHODS

1. Installation on Linux

- Works on most Linux distributions.
- Requires Python 2.7+ (Python 3 recommended)

```
# For RHEL/CentOS
sudo yum install ansible

# For Debian/Ubuntu
sudo apt update
sudo apt install ansible
```

2. Installation on AWS EC2

- Launch an EC2 instance (Amazon Linux/Ubuntu).
- Connect via SSH and install Ansible.

```
# On Amazon Linux
sudo yum install ansible

# On Ubuntu EC2
sudo apt update
sudo apt install ansible
```

3. Installation on Ubuntu

- Use apt package manager.

```
sudo apt update
sudo apt install
software-properties-
common
sudo apt update
sudo apt install ansible
```

4. Installation on RHEL

- Use yum/dnf package manager.

```
# RHEL / CentOS / Rocky /
AlmaLinux
sudo yum install ansible

# For newer versions
sudo dnf install ansible
```

Dependencies

- Python (2.7+/3.x)
- SSH Client
- Sudo/Root privileges (for managed nodes)
- Recommended: pip3 (for latest)

ANSIBLE VERSION VERIFICATION

Check if Ansible is installed successfully.

```
$ ansible --version
```

Output (example):

```
ansible [core 2.16.4]
config file = /etc/ansible/ansible.cfg
configured module search path = ['/usr/share/ansible/plugins/modules',
'/etc/ansible/plugins/modules']
ansible python module location = /usr/lib/python3.9/site-packages/ansible
python version = 3.9.18 (main, Oct  5 2023, 10:11:47)
[GCC 11.4.1 20231218 (Red Hat 11.4.1-4)]
```

CONFIGURATION FILE (ansible.cfg)

Ansible uses a configuration file to control behavior.
Default location:

`/etc/ansible/ansible.cfg`

Check the location used:

```
$ ansible --version | grep config file
```

Key settings in ansible.cfg:

Setting	Description
inventory	Path to inventory file
remote_user	Default SSH user
host_key_checking	Enable/Disable host key checking
retry_files_enabled	Enable/Disable retry files
stdout_callback	Output format (default, yaml, json, etc.)

HANDS-ON COMMANDS

1. Install Ansible (RHEL/CentOS)

```
sudo yum install ansible
```

2. Verify Ansible Installation

```
ansible --version
```

3. Check Ansible Configuration File

```
ansible --version | grep config file
```

INTERVIEW QUESTIONS

Q1. How do you install Ansible?

Ans: Ansible can be installed using the system package manager.
On RHEL/CentOS: `sudo yum install ansible`
On Ubuntu/Debian: `sudo apt install ansible`

Q2. Where is ansible.cfg located?

Ans: The default location is `/etc/ansible/ansible.cfg`.
You can check the exact path using:

```
ansible --version | grep config file
```

DID YOU KNOW?

You can install the latest Ansible using pip:

```
pip3 install --user ansible
# or
pip3 install ansible --upgrade
```

IMPORTANT NOTE

- Ansible runs best on Python 3.
- You only need to install Ansible on the control node.
- Managed nodes do not require any agent installation.

REMEMBER

Ansible is agentless, uses SSH, and needs only Python on managed nodes.

ONE LINE SUMMARY

Install Ansible on the control node, verify the installation, configure ansible.cfg, and start automating infrastructure.

ANSIBLE

INVENTORY MANAGEMENT

By Abhishek Singh

4. INVENTORY MANAGEMENT

Ansible uses inventory files to define the hosts and groups of hosts on which it will work. Inventory can be static (INI/YAML) or dynamic (from external sources).

TOPICS COVERED

- Static Inventory
- Dynamic Inventory
- Inventory Groups
- Host Variables
- Group Variables
- Nested Groups

Practical

Example of a simple static inventory (INI format)

Interview Questions

- What is Inventory?
- Difference between Static and Dynamic Inventory?

INVENTORY CONCEPTS AT A GLANCE

1. Static Inventory

- Defined in INI or YAML files.
- Stored locally.
- Simple to use and maintain.
- Best for small to medium environments.

2. Dynamic Inventory

- Fetched from external sources (AWS, DB, scripts, CMDB, etc.)
- Updates automatically.
- Best for large and cloud environments.

3. Inventory Groups

- Hosts are organized into groups.
- Easy to apply tasks, variables, and configurations.

4. Host Variables

- Variables assigned to specific hosts.
- Override group variables.
- Defined in inventory or separate files.

5. Group Variables

- Variables applied to all hosts in a group.
- Defined in group_vars/ directory or inventory.

6. Nested Groups

- Groups inside other groups.
- Helps in better organization and reuse.

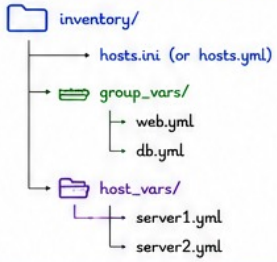
STATIC INVENTORY EXAMPLE (INI FORMAT)

```
# Inventory file: hosts.ini

[web]
server1
server2

[db]
server3
```

INVENTORY FILE STRUCTURE (BEST PRACTICE)



hosts.ini / hosts.yml
Main inventory file.

group_vars/
Variables for groups.

host_vars/
Variables for individual hosts.

EXAMPLE: GROUPS, VARIABLES & NESTED GROUPS

Groups & Nested Groups (INI)

```
[web]
server1
server2

[db]
server3

[servers:children]
web
db
```

Group Variables (group_vars/web.yml)

```
---
http_port: 80
env: production
```

Host Variables (host_vars/server1.yml)

```
---
ansible_host: 10.0.0.1
role: webserver
```

How Ansible Uses Inventory?

- ✓ Reads inventory to know which hosts to manage.
- ✓ Applies tasks, variables, and playbooks to target hosts/groups.
- ✓ Makes automation scalable, reusable and environment-friendly.

DID YOU KNOW?

Inventory can be in INI, YAML, JSON, script, or cloud plugins (AWS, Azure, GCP, etc.).

IMPORTANT NOTE

- Keep inventory simple and organized.
- Use groups and variables to avoid repetition.
- Dynamic inventory is ideal for cloud infrastructure.

REMEMBER

A well-structured inventory is the foundation of scalable Ansible automation.

ONE LINE SUMMARY

Inventory tells Ansible where to work. Organize hosts, groups, and variables well to build scalable and maintainable automation.

ANSIBLE

ANSIBLE AD-HOC COMMANDS

By Abhishek Singh

5. ANSIBLE AD-HOC COMMANDS

Ansible Ad-Hoc Commands are one-line commands used to perform quick automation tasks on managed nodes without creating playbooks.

TOPICS COVERED

- Ping Module
- Command Module
- Shell Module
- Copy Module
- File Module
- Service Module
- Practical Examples
- Interview Questions

AD-HOC MODULES AT A GLANCE

1. Ping Module

- Tests connectivity to managed hosts
- Verifies SSH and Python availability

Example:

```
ansible all -m ping
```

2. Command Module

- Executes commands directly
- Does not support shell operators

Example:

```
ansible web -m command -a "uptime"
```

3. Shell Module

- Executes commands through shell
- Supports pipes, redirects and variables

Example:

```
ansible web -m shell -a "df -h"
```

4. Copy Module

- Copies files from Control Node to Managed Nodes

Example:

```
ansible web -m copy -a "src=index.html dest=/var/www/html/"
```

5. File Module

- Creates, deletes and modifies files/ directories

Example:

```
ansible web -m file -a "path=/tmp/demo state=directory"
```

6. Service Module

- Manages services

Example:

```
ansible web -m service -a "name=httpd state==restarted"
```

PRACTICAL COMMANDS

Check Connectivity	`ansible all -m ping`
Check Disk Usage	`ansible web -m shell -a "df -h"`
Check Memory Usage	`ansible web -m shell -a "free -m"`
Restart Apache Service	`ansible web -m service -a "name=httpd state=restarted"`
Create Directory	`ansible web -m file -a "path=/tmp/test state=directory"`

COMMAND VS SHELL MODULE

Command Module		Shell Module
More Secure	VS	Less Secure
No Shell Processing	VS	Uses Shell
No Pipes Allowed	VS	Supports Pipes
No Redirects	VS	Supports Redirects
Faster Execution	VS	Slightly Slower

INTERVIEW QUESTIONS

Q1. What are Ad-Hoc Commands?

Ans: Ad-Hoc Commands are single-line Ansible commands used to perform quick automation tasks without creating playbooks.

Q2. Difference between Command and Shell Module?

Ans: Command Module executes commands directly without shell features. Shell Module executes commands through a shell and supports pipes, redirects and environment variables.

IMPORTANT NOTE

- Best for quick tasks and troubleshooting.
- Not recommended for complex automation.
- Use Playbooks for repeatable production workflows.
- Uses Inventory file to identify target hosts.

DID YOU KNOW?

You can combine multiple modules in ad-hoc commands to perform powerful tasks instantly!

REMEMBER

Ad-Hoc Commands = Quick Automation
Playbooks = Reusable Automation

EXAMPLE QUICK TIP

Use -m (module) and -a (arguments) to run commands on target hosts instantly.

ONE LINE SUMMARY

Ansible Ad-Hoc Commands provide a fast and simple way to execute automation tasks on managed nodes using built-in modules without creating playbooks.

ANSIBLE

ANSIBLE MODULES

By Abhishek Singh

6. ANSIBLE MODULES

Ansible modules are reusable units of code that Ansible executes on managed nodes to perform specific tasks. Modules make automation simple, consistent and powerful.

TOPICS COVERED

- Core Modules
- Package Modules
- Service Modules
- User Modules
- File Modules
- Cloud Modules
- Common Modules
- Interview Questions

MODULE CATEGORIES

1. Core Modules
- command
- shell
- script
- raw
- setup

2. Package Modules
- yum
- dnf
- apt
- package

3. Service Modules
- service
- systemd

4. User Modules
- user
- group

5. File Modules
- copy
- template
- file
- lineinfile
- blockinfile

6. Cloud Modules
- aws_ec2
- ec2_instance
- s3_bucket
- rds_instance

COMMONLY USED MODULES – EXAMPLES

yum Module

Install packages on RHEL/CentOS

```
ansible web -m yum \
-a "name=httpd state=present"
```

apt Module

Install packages on Ubuntu

```
ansible web -m apt \
-a "name=nginx state=present"
```

service Module

Manage services

```
ansible web -m service \
-a "name=httpd state=started"
```

copy Module

Copy files from control node to managed nodes

```
ansible web -m copy \
-a "src=index.html dest=/var/www/html/"
```

template Module

Deploy Jinja2 templates

```
ansible web -m template \
-a "src=config.j2 dest=/etc/app.conf"
```

file Module

Manage files and directories

```
ansible web -m file \
-a "path=/tmp/demo state=directory"
```

user Module

Manage users

```
ansible web -m user \
-a "name=devops state=present"
```

MODULES AT A GLANCE

Module	Purpose
yum	Install packages (RHEL/CentOS)
apt	Install packages (Debian/Ubuntu)
service / systemd	Manage and control services
copy	Copy files to managed nodes
template	Deploy Jinja2 templates
file	Manage files and directories
user	Manage system users and groups

PRODUCTION USE CASES

Package Installation
- Apache
- Nginx
- Docker
- MySQL

Service Management
- Start services
- Stop services
- Restart services
- Enable/Disable services

User Management
- Create users
- Delete users
- Manage groups
- Set permissions

Configuration Deployment
- Copy config files
- Deploy templates
- Update configs
- Manage files

Cloud Automation
- Create EC2
- Create S3
- Manage RDS
- Attach EBS

INTERVIEW QUESTIONS

Q1. What are modules?

Ans: Modules are reusable units of code that Ansible executes on managed nodes to perform automation tasks.

Q2. Which modules have you used in production?

Ans:
- yum
- apt
- service
- copy
- template
- file
- user
- systemd
- ec2_instance
- s3_bucket
- rds_instance
- command / shell

IMPORTANT NOTE

- Modules are the building blocks of Ansible.
- Every Ansible task uses a module.
- Most modules are idempotent.
- Modules reduce manual effort and improve consistency.

REMEMBER

Playbook + Module = Automation

ONE LINE SUMMARY

Ansible Modules are reusable automation components that perform tasks such as package installation, service management, user administration, file operations, and cloud provisioning.

ANSIBLE

WRITING FIRST PLAYBOOK

By Abhishek Singh

7. WRITING FIRST PLAYBOOK

Ansible Playbooks are YAML files that define a set of tasks to be executed on remote hosts. They are simple, readable and powerful.

TOPICS COVERED

- YAML Basics
- Playbook Structure
- Tasks
- Hosts
- Become
- Example Playbook
- Interview Questions

1. YAML BASICS

- YAML = Yet Another Markup Language
- Indentation is mandatory
- Uses key-value pairs
- Playbooks are written in YAML format

Example:

```
name: nginx
state: present
```

2. PLAYBOOK STRUCTURE

```
---
- hosts: all
  become: yes

  tasks:
    - name: Install Nginx
      yum:
        name: nginx
        state: present
```

- `hosts:` Target hosts defined in inventory
- `become:` Run tasks with elevated privileges
- `tasks:` List of tasks to be executed
- `name:` Name of the task
- `yum:` Module to install package

3. PLAYBOOK COMPONENTS

HOSTS

- Target systems where tasks run.
- Defined in inventory.

Example:

```
hosts: web
```

BECOME

- Executes tasks with elevated privileges.

Example:

```
become: yes
```

TASKS

- Individual actions performed by modules.

Example:

```
tasks:
  - name: Install Nginx
```

4. COMPLETE PLAYBOOK EXAMPLE

```
---
- hosts: all
  become: yes

  tasks:
    - name: Install Nginx
      yum:
        name: nginx
        state: present

    - name: Start Nginx Service
      service:
        name: nginx
        state: started
```

5. PLAYBOOK EXECUTION

Run Playbook:

```
$ ansible-playbook nginx.yml
```

Dry Run (Check mode):

```
$ ansible-playbook nginx.yml --check
```

6. PLAYBOOK FLOW

Inventory (Hosts) → Playbook (YAML) → Tasks (What to do) → Modules (How to do) → Managed Nodes (Execution)

7. INTERVIEW QUESTIONS

Q1. Explain Playbook Structure.

Ans: A Playbook consists of:

- Hosts
- Become
- Variables
- Tasks
- Handlers
- Roles

Playbooks are written in YAML and define the automation workflow executed by Ansible.

Q2. What is a Playbook?

Ans: A Playbook is a YAML file that contains a set of automation tasks executed on managed nodes.

Q3. Why do we use Playbooks instead of Ad-Hoc Commands?

Ans: Playbooks provide reusable, repeatable, version-controlled automation, while Ad-Hoc Commands are suitable only for quick one-time tasks.

8. IMPORTANT NOTE

- Always maintain proper indentation.
- Playbooks are idempotent.
- Store Playbooks in Git repositories.
- Use Roles for large projects.

REMEMBER

Ad-Hoc Commands = Quick Tasks

Playbooks = Production Automation

ONE LINE SUMMARY

ANSIBLE

Ansible Playbooks are YAML-based automation files that define tasks, hosts, and execution flow to automate infrastructure and application management in a repeatable and scalable way.

YAML FUNDAMENTALS

By Abhishek Singh

8. YAML FUNDAMENTALS

YAML (YAML Ain't Markup Language) is a human-readable data serialization format used by Ansible for writing playbooks, variables, configuration files, etc.

Key Points

- Simple and readable
- Indentation is important
- Case sensitive
- Uses key: value pairs

TOPICS COVERED

YAML Syntax | Indentation Rules | Lists | Dictionaries (Mappings) | Variables

1. YAML SYNTAX

- Uses key: value pairs
- Key and value are separated by colon (:)
- Space after colon is required
- Strings do not require quotes (unless needed)

Example:

```yaml
name: nginx
port: 80
enabled: true
path: /etc/nginx/nginx.conf
```

2. INDENTATION RULES

- Indentation is mandatory in YAML
- Use spaces (not tabs)
- Usually 2 spaces per level
- Wrong indentation leads to errors

✓ Correct

```yaml
tasks:
- name: Install Nginx
  yum:
    name: nginx
    state: present
```

✗ Incorrect

```yaml
tasks:
- name: Install Nginx
  yum:
    name: nginx
    state: present
```

3. LISTS

- Lists are items starting with - (dash)
- Used to define multiple values

Example:

```yaml
packages:
  - nginx
  - httpd
  - mysql
  - git
```

4. DICTIONARIES (MAPPINGS)

- Key: Value pairs
- Used to represent structured data

Example:

```yaml
user:
  name: john
  uid: 1001
  shell: /bin/bash
  groups:
    - wheel
    - devops
```

5. VARIABLES

- Variables store reusable values
- Defined in vars section or separate files
- Access using {{ variable_name }}

Example:

```yaml
vars:
  http_port: 80
  app_name: nginx

tasks:
  - name: Install {{ app_name }}
    yum:
      name: "{{ app_name }}"
      state: present
  - name: Start {{ app_name }} service
    service:
      name: "{{ app_name }}"
      state: started
```

YAML DATA TYPES

Type	Example
String	name: nginx
Integer	port: 80
Boolean	enabled: true
List	- item1 - item2
Dictionary	key: value key2: value2

COMMENTS IN YAML

```yaml
# This is a single line comment
# Another comment
```

COMMON YAML MISTAKES

1. Wrong Indentation

✗
```yaml
tasks:
- name: Wrong
  debug:
    msg: hello
```

✓ Correct
```yaml
tasks:
  - name: Correct
    debug:
      msg: hello
```

2. Missing Space After Colon

✗
```yaml
name:nginx
```

✓ Correct
```yaml
name: nginx
```

3. Using Tabs Instead of Spaces

✗
```yaml
tasks:
	- name: Wrong
```

✓ Correct
```yaml
tasks:
  - name: Correct
```

4. Wrong List Format

✗
```yaml
packages:
-nginx
- mysql
```

✓ Correct
```yaml
packages:
  - nginx
  - mysql
```

5. Wrong Dictionary Format

✗
```yaml
name = nginx
port = 80
```

✓ Correct
```yaml
name: nginx
port: 80
```

INTERVIEW QUESTIONS

Q1. Why YAML is used in Ansible?

Ans: YAML is used in Ansible because it is simple, human-readable, easy to write and understand. It allows structured data representation using key-value pairs, lists and dictionaries.

Q2. Common YAML mistakes?

Ans:
- Wrong indentation
- Using tabs instead of spaces
- Missing space after colon
- Wrong list or dictionary format
- Case sensitivity issues

IMPORTANT NOTE

- ✓ YAML is indentation sensitive.
- ✓ Use spaces, not tabs.
- ✓ Keep indentation consistent.
- ✓ Validate YAML before running playbooks.
- ✓ Tools: yamllint, ansible-lint

REMEMBER

Good YAML
=
Error-free Playbooks
+
Happy Automation!

ONE LINE SUMMARY

YAML is the backbone of Ansible playbooks. Mastering YAML syntax, indentation, lists, dictionaries and variables helps in writing efficient and error-free automation.

ANSIBLE

VARIABLES IN ANSIBLE

By Abhishek Singh

9. VARIABLES IN ANSIBLE

Variables in Ansible store data that can be reused in playbooks, templates, and tasks. They make playbooks dynamic, flexible and easier to maintain.

KEY BENEFITS

- ✓ Reusability
- ✓ Maintainability
- ✓ Dynamic Configuration
- ✓ Environment Flexibility

TOPICS COVERED

Host Variables | Group Variables | Playbook Variables | Extra Variables | Facts Variables

TYPES OF VARIABLES

1. HOST VARIABLES

- Variables specific to a single host.
- Defined in inventory file or host_vars/ directory.

Example (host_vars/web.yml)

```
---
http_port: 80
app_name: nginx
```

2. GROUP VARIABLES

- Variables for all hosts in a group.
- Defined in group_vars/ directory.

Example (group_vars/web.yml)

```
---
http_port: 80
app_user: www-data
```

3. PLAYBOOK VARIABLES

- Defined within the playbook under vars: section.

Example

```
---
- hosts: web
  vars:
    http_port: 8080
    app_name: myapp
```

4. EXTRA VARIABLES

- Variables passed at runtime using --extra-vars or -e.

Example

```
ansible-playbook site.yml \
-e "http_port=9000 \
app_name=demo"
```

5. FACTS VARIABLES

- Automatically discovered variables about hosts.
- Gathered by setup module.

Example

```
ansible web -m setup
# ansible_facts['os_family']
# ansible_facts['ipaddress']
```

HOW TO USE VARIABLES IN PLAYBOOK

```
---
- hosts: web
  vars:
    http_port: 80
    app_name: nginx
  tasks:
    - name: Install Nginx
      yum:
        name: "{{ app_name }}"
        state: present
    - name: Start Nginx Service
      service:
        name: "{{ app_name }}"
        state: started
    - name: Show Port
      debug:
        msg: "Nginx will run on port {{ http_port }}"
```

VARIABLE PRECEDENCE (HIGH TO LOW)

HIGH

1. Extra Vars (-e, --extra-vars)
2. Task Vars (vars in a task or include)
3. Block Vars
4. Play Vars (vars in a play)
5. Host Vars (host_vars/)
6. Group Vars (group_vars/)
7. Role Defaults (defaults/main.yml)

LOW

EXAMPLE: PRECEDENCE

```
# group_vars/web.yml
http_port: 80

# host_vars/web1.yml
http_port: 81

# playbook vars
vars:
  http_port: 82

# runtime
ansible-playbook site.yml -e "http_port=83"
```

☆ Final Value of http_port = 83
(From Extra Vars - Highest Priority)

PASSING VARIABLES AT RUNTIME

1. Using -e or --extra-vars

```
ansible-playbook site.yml -e "http_port=8080 app_name=demo"
```

2. Using @file (YAML/JSON file)

- Create a file vars.yml

```
http_port: 8080
app_name: demo
```

- Run the playbook

```
ansible-playbook site.yml -e @vars.yml
```

COMMON VARIABLE SYNTAX

- Access variable: {{ variable_name }}
- Access nested variable: {{ user.name }}
- Access list item: {{ mylist[0] }}
- Default value: {{ variable_name | default('value') }}

Example

```
- debug:
    msg: "App {{ app_name }} will run on port
         {{ http_port | default(80) }}"
```

FACTS EXAMPLE

Facts can be accessed in playbook as:

- ansible_facts['os_family']
- ansible_facts['hostname']
- ansible_facts['default_ipv4']['address']

Example

```
- debug:
    msg: "OS: {{ ansible_facts['os_family'] }}
         IP: {{ ansible_facts['default_ipv4']['address'] }}"
```

INTERVIEW QUESTIONS

Q1. Variable precedence?
Ans: Variables are applied in a specific order. Highest precedence is Extra Vars (-e), followed by task vars, block vars, play vars, host vars, group vars and role defaults (lowest).

Q2. How do you pass variables at runtime?
Ans: Using --extra-vars or -e key=value or by providing a YAML/JSON file using -e @file.yml.

Q3. What are facts variables?
Ans: Facts are automatically discovered variables about managed hosts using the setup module.

IMPORTANT NOTES

- ✓ Always use meaningful variable names.
- ✓ Prefer group_vars/ and host_vars/ for better organization.
- ✓ Variables make playbooks reusable and environment independent.
- ✓ Extra Vars have the highest precedence.

REMEMBER

Good Use of Variables
=
Smart & Reusable Playbooks

ANSIBLE

ONE LINE SUMMARY

Variables in Ansible allow you to store and reuse data, making playbooks dynamic, flexible and easier to manage across different environments.

ANSIBLE FACTS

By Abhishek Singh

10. ANSIBLE FACTS

Ansible Facts are variables automatically discovered about managed hosts by Ansible. Facts provide detailed information about the system, hardware, OS, network, and configuration.

KEY BENEFITS

- Automatic discovery of system information
- Used in playbooks and templates
- Helps in dynamic configuration
- Reduces manual data collection

TOPICS COVERED

Gather Facts

System Information

Hardware Details

OS Information

Commands & Examples

1. GATHER FACTS

- Ansible automatically gathers facts from all managed hosts.
- Facts are collected using the setup module.
- Fact gathering is enabled by default.
- Can be disabled if not required.

In ansible.cfg

```
[defaults]
gathering = smart   # or yes / no
```

2. GET FACTS USING SETUP MODULE

Command

```
ansible all -m setup
```

Example Output (Partial)

```
webserver | SUCCESS => {
    "ansible_facts": {
        "ansible_hostname": "webserver",
        "ansible_os_family": "RedHat",
        "ansible_distribution": "Rocky",
        "ansible_distribution_major_version": "9",
        "ansible_kernel": "5.14.0-427.40.1.el9_4.x86_64",
        "ansible_processor_vcpus": 2,
        "ansible_memtotal_mb": 2048,
        "ansible_default_ipv4": {
            "address": "192.168.1.10",
            "gateway": "192.168.1.1"
        }
    }
}
```

3. TYPES OF FACTS

System Information	Hostname, Kernel, Architecture, Python Version, FQDN
Hardware Details	CPU, Memory, Disk, BIOS, Virtualization
OS / Distribution	OS Family, Distribution, Version, Release, Platform
Network Information	IP Address, MAC Address, Gateway, DNS, Interfaces
Other Information	Environment Variables, Mounts, Packages, Services

4. USE FACTS IN PLAYBOOK

```
- hosts: all
  gather_facts: yes

  tasks:
    - name: Print OS Family
      debug:
        var: ansible_os_family

    - name: Print IP Address
      debug:
        var: ansible_default_ipv4.address
```

5. COMMON FACTS EXAMPLES

Fact	Description
ansible_hostname	Hostname of the system
ansible_os_family	OS Family (RedHat/Debian/etc)
ansible_distribution	Distribution Name (Rocky, Ubuntu, etc)
ansible_distribution_version	Distribution Version
ansible_kernel	Kernel Version
ansible_processor_vcpus	Number of CPU Cores
ansible_memtotal_mb	Total Memory (in MB)
ansible_default_ipv4.address	Primary IP Address

6. DISABLE FACT GATHERING

In Playbook

```
- hosts: all
  gather_facts: no
```

In Command

```
ansible-playbook site.yml -e "gather_facts=false"
```

7. QUICK FACTS COMMANDS

Get all facts for all hosts	`ansible all -m setup`
Get facts for a group	`ansible web -m setup`
Get facts for a single host	`ansible server1 -m setup`
Get specific fact using grep	`ansible all -m setup | grep ansible_hostname`

INTERVIEW QUESTIONS

Q1. What are Facts?
Ans: Facts are variables automatically discovered by Ansible about managed hosts using the setup module.

Q2. How are Facts useful?
Ans: Facts provide system, hardware, OS and network information which can be used in playbooks, templates and conditional statements for dynamic automation.

Q3. Can we disable fact gathering?
Ans: Yes, fact gathering can be disabled in ansible.cfg or in playbooks using gather_facts: no.

IMPORTANT NOTES

- Facts are gathered at the start of a play.
- Fact gathering can slow down execution on large infrastructures.
- Use fact caching to improve performance.
- Facts are stored in memory during play execution.
- Use facts to make playbooks dynamic and reusable.

REMEMBER

Facts = Real-time Information about Managed Hosts

Better Facts, Better Automation!

ONE LINE SUMMARY

Ansible Facts automatically collect detailed information about managed hosts which helps in building dynamic, intelligent and efficient automation.

CONDITIONALS IN ANSIBLE

By Abhishek Singh

11. CONDITIONALS

Conditionals allow you to run tasks only when specific conditions are met. This makes playbooks dynamic, intelligent and environment-aware.

KEY BENEFITS

- Execute tasks only when needed
- Support complex logic
- OS and environment aware
- Improves efficiency and reusability

TOPICS COVERED

- when Statement
- if Conditions
- OS Based Execution
- Multiple Conditions
- Conditional Operators
- Real-Time Examples
- Interview Questions

CONDITIONALS AT A GLANCE

1. when Statement

Used to execute a task only when a condition is true.

```yaml
- name: Install Nginx
  yum:
    name: nginx
    state: present
  when: ansible_os_family == "RedHat"
```

2. If Conditions

```yaml
- name: Restart Service
  service:
    name: nginx
    state: restarted
  when: service_status == "running"
```

3. OS Based Execution

```yaml
- name: Install Apache on RHEL
  yum:
    name: httpd
    state: present
  when: ansible_os_family == "RedHat"

- name: Install Apache on Ubuntu
  apt:
    name: apache2
    state: present
  when: ansible_os_family == "Debian"
```

4. Multiple Conditions

AND Condition

```yaml
when:
  - ansible_os_family == "RedHat"
  - ansible_distribution_major_version == "9"
```

OR Condition

```yaml
when: ansible_os_family == "RedHat"
  or ansible_os_family == "Debian"
```

CONDITIONALL OPERATORS

Operator	Meaning
==	Equal To
!=	Not Equal To
>	Greater Than
<	Less Than
>=	Greater Than Equal
<=	Less Than Equal
and	Logical AND
or	Logical OR
not	Logical NOT

REAL-TIME USE CASES

- Install Package Based On OS
- Execute Task Only If Service Is Running
- Run Backup On Production Servers
- Skip Tasks On Development Environment
- Deploy Application Based On Version

PRACTICAL EXAMPLES

1. Check Memory Before Execution

```yaml
- name: Run Task
  debug:
    msg: "Enough Memory Available"
  when: ansible_memtotal_mb > 2048
```

2. Run Task Only For Production

```yaml
- name: Deploy Application
  shell: ./deploy.sh
  when: env == "prod"
```

INTERVIEW QUESTIONS

Q1. How do you run tasks conditionally?

Ans: Ansible uses the **when** statement to execute tasks conditionally. A task runs only when the specified condition evaluates to true.

Q2. Can you use multiple conditions in Ansible?

Ans: Yes. Multiple conditions can be combined using **and**, **or**, and **not** operators.

Q3. How do you execute tasks based on OS type?

Ans: Using Ansible Facts such as:

```yaml
when: ansible_os_family == "RedHat"
```

IMPORTANT NOTE

- Conditions are evaluated at runtime.
- Facts are commonly used with conditions.
- Supports AND, OR, and NOT operations.
- Makes playbooks dynamic and reusable.

REMEMBER

Conditionals
=
Smart Automation

Without Conditions
=
Same Task Everywhere

With Conditions
=
Right Task on Right Server

ONE LINE SUMMARY

Ansible Conditionals use the when statement to execute tasks dynamically based on operating systems, variables, facts, and custom conditions, making automation intelligent and flexible.

LOOPS IN ANSIBLE

By Abhishek Singh

12. LOOPS

Loops in Ansible allow you to run a task multiple times for a list of items. They help in reducing code duplication and making playbooks simple and efficient.

KEY BENEFITS

- Reduce repetitive tasks
- Save time and effort
- Improve readability
- Easier maintenance
- Supports bulk operations

TOPICS COVERED

loop | with_items | Iterations | Multiple Resource Creation | Real-Time Use Cases | Interview Questions

LOOPS AT A GLANCE

1. loop

Modern method for iterating over a list of items.

Example:

```
- name: Install Packages
  yum:
    name: "{{ item }}"
    state: present
  loop:
    - nginx
    - git
    - docker
```

2. with_items

Legacy looping method.

Example:

```
- name: Create Users
  user:
    name: "{{ item }}"
    state: present
  with_items:
    - devops
    - admin
    - testuser
```

3. Iterations

Loops allow the same task to run multiple times using different values.

Example:

```
- name: Create Directories
  file:
    path: "/app/{{ item }}"
    state: directory
  loop:
    - logs
    - backup
    - config
```

4. Multiple Resource Creation

Create multiple resources with a single task.

Example:

```
- name: Create Users
  user:
    name: "{{ item }}"
    state: present
  loop:
    - john
    - mike
    - david
```

COMMON LOOP EXAMPLES

Install Multiple Packages

```
- name: Install Packages
  yum:
    name: "{{ item }}"
    state: present
  loop:
    - nginx
    - git
    - httpd
    - docker
```

Start Multiple Services

```
- name: Start Services
  service:
    name: "{{ item }}"
    state: started
  loop:
    - nginx
    - docker
```

Create Multiple Files

```
- name: Create Files
  file:
    path: "/tmp/{{ item }}"
    state: touch
  loop:
    - file1.txt
    - file2.txt
    - file3.txt
```

LOOP VS WITH_ITEMS

loop	VS	with_items
Recommended	👍	Legacy
Cleaner Syntax	👍	Older Syntax
More Flexible	👍	Limited
Preferred in New Playbooks	👍	Backward Compatibility

QUICK SYNTAX REMINDER

```
loop:
  - item1
  - item2
  - item3
```

REAL-TIME USE CASES

Package Installation
- nginx
- docker
- git
- java

User Creation
- DevOps Users
- Application Users
- Service Accounts

Service Management
- Start Services
- Stop Services
- Restart Services

File & Directory Management
- Create Directories
- Copy Files
- Deploy Configurations

INTERVIEW QUESTIONS

Q1. How do you install multiple packages using loops?

Ans:

```
- name: Install Packages
  yum:
    name: "{{ item }}"
    state: present
  loop:
    - nginx
    - git
    - docker
```

Ansible executes the same task for each item in the loop list.

Q2. What is the difference between loop and with_items?

Ans:
- loop is the modern and recommended approach.
- with_items is the older looping syntax maintained for compatibility.

Q3. Why use loops in Ansible?

Ans: Loops reduce code duplication and make playbooks simpler, reusable, and easier to maintain.

IMPORTANT NOTE

- Use loop instead of with_items in modern playbooks.
- Loops improve readability.
- Can be combined with conditions and variables.
- Useful for bulk resource creation.

REMEMBER

Without Loops
=
Repeated Tasks

With Loops
=
Smart Automation

ONE LINE SUMMARY

Ansible Loops allow a single task to execute multiple times for different items, making automation scalable, reusable, and efficient.

ANSIBLE

HANDLERS IN ANSIBLE

By Abhishek Singh

13. HANDLERS

Handlers are special tasks that run only when notified by other tasks. They are typically used for actions like restarting services, reloading configurations, etc.

KEY BENEFITS

- Run only when changes occur
- Improve efficiency
- Avoid unnecessary restarts
- Used for service management
- Better automation and performance

TOPICS COVERED

- notify (Notify Keyword)
- Handlers (Block)
- Service Restart Automation
- Real-Time Examples
- Interview Questions

HOW HANDLERS WORK

Task Makes a Change → Example: Update Configuration File

Notify Handler → Using notify keyword

Handler Executes → Handler runs at the end of the play

Service Restarted / Reloaded → Service restart or reload automatically

1. EXAMPLE: USING NOTIFY

```yaml
- name: Update Nginx Configuration
  template:
    src: nginx.conf.j2
    dest: /etc/nginx/nginx.conf
  notify: Restart Nginx
```

The handler "Restart Nginx" will run only if the above task changes the file.

2. HANDLERS SECTION

```yaml
handlers:
 - name: Restart Nginx
   service:
     name: nginx
     state: restarted
```

3. COMPLETE PLAYBOOK EXAMPLE

```yaml
---
- hosts: all
  become: yes
  tasks:
  - name: Update Nginx Config
    template:
      src: nginx.conf.j2
      dest: /etc/nginx/nginx.conf
    notify: Restart Nginx
  handlers:
    - name: Restart Nginx
      service:
        name: nginx
        state: restarted
```

4. MULTIPLE NOTIFY & HANDLERS

```yaml
tasks:
 - name: Update App Config
   template:
     src: app.conf.j2
     dest: /etc/app/app.conf
   notify: Restart App

 - name: Update Nginx Config
   template:
     src: nginx.conf.j2
     dest: /etc/nginx/nginx.conf
   notify: Restart Nginx

handlers:
 - name: Restart App
   service:
     name: app
     state: restarted

 - name: Restart Nginx
   service:
     name: nginx
     state: restarted
```

HANDLER BEHAVIOR

- Handlers run once per play even if notified multiple times.
- Handlers run after all tasks in the play are completed.
- If a task doesn't make any change, handler will not run.
- Use handlers for service restart, reload, cleanup, etc.

WHEN TO USE HANDLERS?

- Restarting services
- Reloading configurations
- Clearing cache
- Reapplying firewall rules
- Any action after configuration change

TASK vs HANDLER

Task	VS	Handler
Runs immediately		Runs at the end of the play
Runs every time the play runs		Runs only when notified
Always executed		Executed conditionally
Used for general actions		Used for service/reload actions
No dependency		Dependent on notify

BEST PRACTICES

- Always use notify for handlers.
- Keep handler names meaningful.
- Handlers should be idempotent.
- Use handlers only for actions that should run on change.

REAL-TIME USE CASE

```yaml
- name: Deploy Application Config
  template:
    src: app.conf.j2
    dest: /etc/app/app.conf
  notify: Restart Application

handlers:
 - name: Restart Application
   service:
     name: app
     state: restarted
```

Application will restart only if config file is changed.

INTERVIEW QUESTIONS

Q1. What are Handlers?
Ans: Handlers are special tasks that run only when notified by other tasks. They are used for actions like service restart, reload, or cleanup.

Q2. Difference between Task and Handler?
Ans: Tasks run every time a playbook runs, whereas handlers run only when notified by a task and only once at the end of the play.

Q3. Why do we use Handlers?
Ans: To improve efficiency and performance by running actions only when changes occur.

IMPORTANT NOTES

- Handlers improve efficiency.
- They run once per play.
- Use notify to trigger handlers.
- Perfect for service restart automation.

REMEMBER

Handlers = Smart Actions on Change

No Change = No Action

Change Detected = Handler Runs

ONE LINE SUMMARY

Handlers in Ansible run only when notified by tasks on change, making automation efficient by restarting or reloading services only when needed.

TEMPLATES (JINJA2) IN ANSIBLE

By Abhishek Singh

14. TEMPLATES (JINJA2)

Ansible Templates allow you to create dynamic configuration files using Jinja2 syntax. Variables and facts are inserted into templates and rendered on the target host.

KEY BENEFITS

- Create dynamic configuration files
- Use variables and facts
- Easier management of configs
- Reusability and flexibility
- Avoid hardcoding values

TOPICS COVERED

- Jinja2 Syntax
- Dynamic Configuration Files
- Variables in Templates
- Practical Examples
- Best Practices
- Interview Questions

HOW TEMPLATES WORK

1. JINJA2 SYNTAX BASICS

Jinja2 uses special delimiters:

- {{ variable }} → Print variable
- {% statement %} → Logic/Statements
- {# comment #} → Comment

Examples:

```
{{ variable_name }}          # Output variable
{{ ansible_hostname }}       # Output fact
{% if variable %}            # If condition
{% for item in items %}      # Loop
{# This is a comment #}      # Comment
```

2. EXAMPLE TEMPLATE FILE (nginx.conf.j2)

```
server {
    listen {{ nginx_port }};
    server_name {{ server_name }};

    location / {
        root {{ doc_root }};
        index index.html index.htm;
    }
    {% if enable_ssl %}
     listen 443 ssl;
     ssl_certificate {{ ssl_cert }};
     ssl_certificate_key {{ ssl_key }};
    {% endif %}
}
```

3. USE TEMPLATE MODULE IN PLAYBOOK

```
- name: Deploy Nginx Config using Template
  template:
    src: nginx.conf.j2
    dest: /etc/nginx/nginx.conf
    owner: root
    group: root
    mode: '0644'
  notify: Restart Nginx

handlers:
  - name: Restart Nginx
    service:
      name: nginx
      state: restarted
```

4. VARIABLES USED IN TEMPLATE

Variable	Description
nginx_port	Port on which Nginx will listen
server_name	Domain name of the server
doc_root	Document root directory
enable_ssl	Enable or disable SSL
ssl_cert	Path to SSL certificate
ssl_key	Path to SSL private key

5. TEMPLATE RENDERING EXAMPLE

Variables:

```
nginx_port: 80
server_name: example.com
doc_root: /var/www/html
enable_ssl: true
ssl_cert: /etc/ssl/certs/example.crt
ssl_key: /etc/ssl/private/example.key
```

Rendered Output (Final File on Target):

```
server {
    listen 80;
    server_name example.com;

    root /var/www/html;
    index index.html index.htm;
}

    listen 443 ssl;
    ssl_certificate /etc/ssl/certs/example.crt;
    ssl_certificate_key /etc/ssl/private/example.key;
}
```

6. COMMON USE CASES

- Generate configuration files dynamically
- Deploy environment-specific configurations
- Insert variables, facts, and conditionals
- Manage application configs (Nginx, Apache, MySQL, etc.)
- Maintain single template for multiple servers

JINJA2 CONTROL STRUCTURES

Condition:

```
{% if variable == "value" %}
  ...
{% elif variable == "other" %}
  ...
{% else %}
  ...
{% endif %}
```

Loop:

```
{% for item in items %}
  {{ item }}
{% endfor %}
```

BEST PRACTICES

- ✓ Keep templates in the templates/ directory
- ✓ Use meaningful variable names
- ✓ Use conditionals to handle different scenarios
- ✓ Test templates before deploying
- ✓ Use handlers to restart services only if needed

INTERVIEW QUESTIONS

Q1. What is Jinja2?
Ans: Jinja2 is a templating engine used by Ansible to create dynamic configuration files. It allows variables, loops, conditions, and filters in templates.

Q2. Why use templates in Ansible?
Ans: Templates help generate dynamic configuration files based on variables and facts, making playbooks flexible, reusable, and easier to manage.

Q3. Where should template files be stored?
Ans: Template files should be stored in the templates/ directory of your Ansible role or playbook.

IMPORTANT NOTES

- Templates are rendered on the target host.
- Use {{ }} for expressions and {% %} for logic.
- Variables can come from inventory, vars_files, or facts.
- Handlers can be used to take action after template changes.

REMEMBER

Templates + Variables = Dynamic Configurations

One Template, Multiple Environments!

ONE LINE SUMMARY

Ansible Templates (Jinja2) allow you to create dynamic, flexible configuration files using variables, facts, loops, and conditions for smart automation.

ANSIBLE

ROLES IN ANSIBLE

By Abhishek Singh

15. ROLES

Roles in Ansible help you organize playbooks into a reusable, modular and maintainable structure. They promote reusability and follow best practices.

KEY BENEFITS

- Reusability and modularity
- Standardized structure
- Easy to share and maintain
- Simplifies complex playbooks
- Encourages best practices

TOPICS COVERED

Role Structure | tasks | handlers | defaults | vars | templates | files | Interview Questions

ROLE STRUCTURE

A role has a specific directory structure.

```
myrole/
├── defaults/
│   └── main.yml
├── files/
├── handlers/
│   └── main.yml
├── meta/
│   └── main.yml
├── tasks/
│   └── main.yml
├── templates/
└── vars/
    └── main.yml
└── README.md
```

Directory	Purpose
Defaults/	Default variables
files/	Static files to be copied
handlers/	Handlers (operations triggered by notify)
meta/	Role metadata and dependencies
tasks/	Main task definitions
templates/	Jinja2 template files
vars/	Role specific variables
README.md	Role documentation

DIRECTORY EXPLANATION

- tasks/ — Contains the main tasks (main.yml). This is the core of the role.
- handlers/ — Contains handlers that are notified by tasks.
- defaults/ — Defines default variables with low precedence.
- vars/ — Defines variables with higher precedence.
- templates/ — Stores Jinja2 template files for dynamic config.
- files/ — Stores files that will be copied to remote hosts.
- meta/ — Contains role dependencies and meta info.

EXAMPLE: ROLE USAGE IN PLAYBOOK

```yaml
---
- hosts: webservers
  become: yes
  roles:
    - nginx
    - common
    - firewall
```

ROLE IN PLAYBOOK

- Include multiple roles easily
- Keeps playbooks clean
- Promotes reusability
- Easy to maintain and share

INSIDE ROLE DIRECTORIES (EXAMPLES)

tasks/main.yml

```yaml
---
- name: Install Nginx
  yum:
    name: nginx
    state: present
  notify: Restart Nginx

- name: Copy Config
  template:
    src: nginx.conf.j2
    dest: /etc/nginx/
      nginx.conf
```

Defines tasks to be executed.

handlers/main.yml

```yaml
---
- name: Restart Nginx
  service:
    name: nginx
    state: restarted
```

Contains handlers that run on notification.

defaults/main.yml

```yaml
---
nginx_port: 80
nginx_user: www-data
```

Default variables with low precedence.

vars/main.yml

```yaml
---
nginx_port: 8080
nginx_user: nginx
```

Role variables with higher precedence.

templates/example.j2

```
server {
  listen {{ nginx_port }};
  server_name {{ domain }};
   root /var/www/html;
}
```

Jinja2 template for dynamic configuration.

files/

logo.png
app.conf
script.sh

Static files to be copied to remote host.

HOW TO CREATE A ROLE

1. Create Role
   `ansible-galaxy init myrole`
2. Add Tasks, Handlers, Vars, Templates, Files
3. Use Role in Playbook

```yaml
roles:
  - myrole
```

REAL-TIME USE CASES

- Web Server Setup (Nginx/Apache)
- Database Installation (MySQL/PostgreSQL)
- User & Permission Management
- Security Hardening (Firewall, SSH, Fail2ban)
- Application Deployment
- Monitoring Stack Installation

ROLE VS TASK FILE

Role	VS	Task File
Reusable		Not Reusable
Modular Structure		Single File
Easy to Maintain		Hard to Maintain
Shareable		Not Shareable
Best Practice		Ad-hoc Use

BEST PRACTICES

- Keep roles small and focused
- Use meaningful role names
- Document roles (README.md)
- Use defaults for variables
- Use handlers for service restarts
- Follow standard directory structure

INTERVIEW QUESTIONS

Q1. What are Roles?
Ans: Roles in Ansible are a way to organize playbooks into a standardized directory structure for reusability, maintainability and sharing.

Q2. What are the benefits of using Roles?
Ans: Roles promote reusability, modularity, code organization, maintainability, and make it easy to share and collaborate.

Q3. How do you use a Role in a Playbook?
Ans: By using the roles keyword and listing the required roles.

```yaml
roles:
  - myrole
```

IMPORTANT NOTES

- Roles follow a standard and predictable structure.
- Variables in vars/ override defaults/.
- Handlers run only when notified.
- Roles can have dependencies.
- Use roles to keep playbooks clean and scalable.

REMEMBER

Roles = Reusability
+
Modularity
+
Maintainability
=
Better Automation!

ONE LINE SUMMARY

Roles in Ansible help you build reusable, modular and maintainable automation by organizing tasks, variables, templates, and handlers in a standard structure.

ANSIBLE

ANSIBLE GALAXY

By Abhishek Singh

16. ANSIBLE GALAXY

Ansible Galaxy is the official hub for finding, sharing and reusing roles and collections. It helps you speed up automation by using community contributions.

KEY BENEFITS

- Reuse existing roles and collections
- Save development time
- Promote best practices
- Community driven content
- Easy to install and use

TOPICS COVERED

Reusable Roles | Galaxy Installation | Collections | Commands | Examples | Interview Questions

HOW ANSIBLE GALAXY WORKS

Search on Ansible Galaxy → Install Roles / Collections → Use in Playbooks → Run Playbook → Automation Success

1. ANSIBLE GALAXY OVERVIEW

- Ansible Galaxy is a content hub.
- Provides thousands of roles and collections.
- Hosted at: https://galaxy.ansible.com
- You can publish your own roles and collections.

2. GALAXY INSTALLATION

Ansible Galaxy comes pre-installed with Ansible.

```
ansible-galaxy --version

# To upgrade galaxy
ansible-galaxy collection install
ansible.builtin --upgrade
```

3. REUSABLE ROLES

Use community roles to avoid writing from scratch.

Example Role:
geerlingguy.nginx

4. INSTALL A ROLE

Install a role from Ansible Galaxy.

```
ansible-galaxy install geerlingguy.nginx
```

By default, roles are installed in:
~/.ansible/roles/

5. INSTALL A ROLE (SPECIFY PATH)

Install role in a specific directory.

```
ansible-galaxy install geerlingguy.nginx \
  -p ./roles
```

6. SEARCH FOR ROLES

Search roles on Galaxy.

```
ansible-galaxy search nginx
```

7. COLLECTIONS

- Collections are a way to distribute content.
- They can include modules, plugins, and roles.
- Collections follow namespaces.
- Example: community.general, ansible.netcommon

8. INSTALL A COLLECTION

Install a collection from Galaxy.

```
ansible-galaxy collection install
community.general
```

9. LIST INSTALLED CONTENT

List installed roles and collections.

```
# List installed roles
ansible-galaxy role list

# List installed collections
ansible-galaxy collection list
```

10. EXAMPLE: INSTALL AND USE A ROLE

Install Role

```
ansible-galaxy install geerlingguy.nginx
```

Use in Playbook

```
---
- hosts: webservers
  become: yes
  roles:
    - geerlingguy.nginx
```

11. COMMON GALAXY COMMANDS

Command	Description
ansible-galaxy search <name>	Search roles on Galaxy
ansible-galaxy install <role>	Install a role
ansible-galaxy install <role> -p <path>	Install role in specific path
ansible-galaxy role list	List installed roles
ansible-galaxy collection install <name>	Install a collection
ansible-galaxy collection list	List installed collections
ansible-galaxy collection search <name>	Search collections

12. ROLE DIRECTORY STRUCTURE

```
myrole/
├── defaults/      # Default variables
├── files/         # Static files
├── handlers/      # Handlers
├── meta/          # Role metadata
├── tasks/         # Main tasks
├── templates/     # Jinja2 templates
├── vars/          # Role variables
└── README.md      # Documentation
```

INTERVIEW QUESTIONS

Q1. What is Ansible Galaxy?
Ans: Ansible Galaxy is the official hub for finding, sharing and using Ansible roles and collections.

Q2. Why use Ansible Galaxy?
Ans: It saves time, promotes best practices and allows reuse of community-tested content.

Q3. How do you install a role from Ansible Galaxy?
Ans: Use the command:

```
ansible-galaxy install <role_name>
```

IMPORTANT NOTES

- Galaxy helps in reusing tested and trusted content.
- Prefer roles and collections from trusted authors.
- Always check the documentation and compatibility.
- Keep your Galaxy content updated.

REMEMBER

Galaxy = Reuse
Reuse = Save Time
Save Time = Better Automation

ONE LINE SUMMARY

Ansible Galaxy is the official hub to find, install and use reusable roles and collections to automate faster with trusted community content.

Page 17

DYNAMIC INVENTORY IN ANSIBLE

By Abhishek Singh

17. DYNAMIC INVENTORY

Dynamic Inventory allows Ansible to automatically discover and manage hosts from cloud providers. It eliminates the need to maintain static inventory files.

KEY BENEFITS

- Automatic host discovery
- Always up-to-date
- Scales with infrastructure
- Reduced manual effort
- Perfect for cloud environments

TOPICS COVERED

- Dynamic Inventory Overview
- AWS Dynamic Inventory
- Azure Dynamic Inventory
- GCP Dynamic Inventory
- Plugins & Configuration
- Commands & Examples
- Interview Questions

HOW DYNAMIC INVENTORY WORKS

Query Cloud Provider API → Discover Resources → Generate Inventory → Use in Playbooks → Always Up-to-Date

1. AWS DYNAMIC INVENTORY

- Uses AWS EC2 API to fetch instances.
- Plugin: `amazon.aws.aws_ec2`

Example Configuration (aws_ec2.yml)

```yaml
plugin: amazon.aws.aws_ec2
regions:
  - us-east-1
  - us-west-2
filters:
    instance-state-name: running
keyed_groups:
  - key: tags.Name
    prefix: name
```

EC2 Instances · Auto Scaling · Tag Based Groups

2. AZURE DYNAMIC INVENTORY

- Uses Azure Resource Manager API.
- Plugin: `azure.azcollection.azure_rm`

Example Configuration (azure_rm.yml)

```yaml
plugin: azure.azcollection.azure_rm
include_vm_resource_groups:
  - myResourceGroup
auth_source: auto
include_vm_sizes: true
include_public_ip_addresses: true
```

Virtual Machines · Resource Groups · Tags & Filters

3. GCP DYNAMIC INVENTORY

- Uses Google Cloud Compute API.
- Plugin: `google.cloud.gcp_compute`

Example Configuration (gcp_compute.yml)

```yaml
plugin: google.cloud.gcp_compute
projects:
  - my-gcp-project
zones:
  - us-central1-a
filters:
  status: RUNNING
```

Compute Instances · Zones · Labels & Filters

INVENTORY PLUGINS

Ansible provides built-in inventory plugins for major cloud providers.

- amazon.aws.aws_ec2 (AWS)
- azure.azcollection.azure_rm (Azure)
- google.cloud.gcp_compute (GCP)
- openstack.cloud.openstack (OpenStack)
- vmware.vmware.vmware (VMware)

USEFUL COMMANDS

```
ansible-inventory -i aws_ec2.yml --list
```
▶ List all hosts from dynamic inventory

```
ansible-inventory -i aws_ec2.yml --graph
```
▶ Show inventory in graph format

```
ansible-inventory -i azure_rm.yml --host <hostname>
```
▶ Show variables for a host

```
ansible-inventory -i gcp_compute.yml --list
```
▶ List hosts from GCP

EXAMPLE PLAYBOOK USING DYNAMIC INVENTORY

```yaml
---
- name: Ping all cloud instances
  hosts: all
  gather_facts: no
  tasks:
   - name: Ping Hosts
     ping:
```

Make sure you have proper credentials/roles configured for your cloud provider.

REAL-TIME USE CASES

- Auto scale EC2 instances and manage with Ansible instantly.
- Manage Azure VMs across subscriptions.
- Manage GCP instances across projects & zones.
- Apply configurations only to running resources.
- Avoid manual inventory updates.

STATIC vs DYNAMIC INVENTORY

Feature	Static Inventory	Dynamic Inventory
Host Management	Manual	Automatic
Updates	Manual Updates	Real-time Updates
Scalability	Limited	Highly Scalable
Cloud Integration	Not Built-in	Built-in
Maintenance	High	Low
Best For	Small Environments	Cloud Environments

BEST PRACTICES

- Use least privilege IAM roles.
- Enable caching to reduce API calls.
- Use filters to include only required resources.
- Group hosts using tags, labels, or metadata.
- Test inventory with --list before running playbooks.

INTERVIEW QUESTIONS

Q1. What is Ansible Dynamic Inventory?
Ans: Dynamic Inventory automatically discovers hosts from external sources like cloud providers and updates inventory in real time.

Q2. How do you manage dynamic cloud infrastructure?
Ans: By using dynamic inventory plugins to fetch cloud resources and running playbooks on them automatically.

Q3. What are the benefits of Dynamic Inventory?
Ans: Automatic discovery, real-time updates, scalability, and reduced manual effort.

IMPORTANT NOTES

- Dynamic inventory requires cloud credentials.
- Inventory data is fetched at runtime.
- Supports filters, groups and keyed groups.
- Always secure your credentials.
- Use caching for better performance.

REMEMBER

Dynamic Inventory = Smart Discovery + Automation + Scalability

ONE LINE SUMMARY

Dynamic Inventory in Ansible automatically discovers and manages cloud resources in real time, making automation scalable, efficient, and maintenance-free.

ANSIBLE VAULT

SECURE YOUR SECRETS

By Abhishek Singh

18. VAULT

Ansible Vault is used to encrypt sensitive data such as passwords, tokens, private keys and other secrets in your Ansible files. It ensures your secrets remain secure in version control.

KEY BENEFITS

- Encrypt sensitive data
- Keep secrets out of plain text
- Secure credentials in playbooks
- Easy encryption & decryption
- Password or key based security

TOPICS COVERED

Encrypt Secrets | Password Files | Secure Credentials | Commands | Playbook Integration | Best Practices | Interview Questions

HOW ANSIBLE VAULT WORKS

Create / Edit Secret File → Encrypt with Ansible Vault → Store Encrypted File in Repo → Use in Playbooks → Decrypt at Runtime (With Password)

ESSENTIAL ANSIBLE VAULT COMMANDS

Command	Description
`ansible-vault create secret.yml`	Create a new encrypted file.
`ansible-vault edit secret.yml`	Edit an encrypted file.
`ansible-vault view secret.yml`	View an encrypted file.
`ansible-vault encrypt file.yml`	Encrypt an existing file.
`ansible-vault decrypt file.yml`	Decrypt an encrypted file.
`ansible-vault rekey file.yml`	Change the password of an encrypted file.
`ansible-vault encrypt_string 'mypassword'`	Encrypt a string.
`ansible-vault --version`	Check Ansible Vault version.

CREATE SECRET FILE EXAMPLE

```
$ ansible-vault create secret.yml
New Vault password:
Confirm New Vault password:

# secret.yml (encrypted content)
$ANSIBLE_VAULT;1.1;AES256
653864623031633738366231613733363361...
366533633633623800335664323963303663...
```

The file content is encrypted and can only be decrypted using the correct password.

ENCRYPT STRING EXAMPLE

```
$ ansible-vault encrypt_string 'mypassword'
New Vault password:
Confirm New Vault password:

!vault |
$ANSIBLE_VAULT;1.1;AES256
386381663461343282653221264344336131833136661...
643266336235646137643639366130633266...
```

Use this encrypted string in vars or templates.

USE VAULT IN PLAYBOOK

```
- name: Use Vault Variables
  hosts: all
  vars_files:
    - secret.yml

  tasks:
   - name: Print Secret
     debug:
       msg: "The secret password is {{ db_password }}"
```

PASSWORD FILE (RECOMMENDED)

Store the vault password in a file for non-interactive use.

1. Create a password file
   ```
   $ echo 'MyVaultPass' > .vault_pass.txt
   $ chmod 600 .vault_pass.txt
   ```
2. Use password file with vault
   ```
   $ ansible-playbook playbook.yml --vault-password-file .vault_pass.txt
   ```

Improves security by avoiding typing password every time.

HOW TO DECRYPT A FILE

```
$ ansible-vault decrypt secret.yml
Vault password:

# File is now decrypted.
```

Re-encrypt the file after editing if needed.

REKEY (CHANGE PASSWORD)

```
$ ansible-vault rekey secret.yml
Vault password (old):
New Vault password:
Confirm New Vault password:

# Password updated successfully.
```

WHERE TO USE VAULT

- Database passwords
- API tokens & keys
- Private keys / Certificates
- Cloud access credentials
- Any sensitive configuration

FILE EXTENSION

Vault does not change file extension.
Any filetype can be encrypted.

Examples:
- vars/dev.yml
- group_vars/all.yml
- files/config.txt
- templates/app.conf.j2

VAULT PASSWORD vs PASSWORD FILE

Vault Password	Password File
Entered manually	Stored in a file
Less secure for automation	More secure for automation
Good for simple use	Recommended for production
Not version controlled	File with restricted permissions (600)

BEST PRACTICES

- Always use Password File for automation.
- Keep password file outside version control.
- Restrict file permissions (chmod 600).
- Use vault for all sensitive data.
- Rotate passwords regularly.

SECURITY TIPS

- Never commit vault password file.
- Use strong passwords.
- Limit access to vault files.
- Audit who can access secrets.
- Use CI/CD secrets management.

INTERVIEW QUESTIONS

Q1. What is Ansible Vault?
Ans: Ansible Vault is a feature that allows you to encrypt sensitive data such as passwords, tokens, and keys in Ansible files.

Q2. How do you secure passwords in Ansible?
Ans: By encrypting files or strings with Ansible Vault and using password files to decrypt them during playbook execution.

Q3. Can vault files be edited?
Ans: Yes, using "ansible-vault edit <files>" command.

IMPORTANT NOTES

- Vault works on any text file.
- Always back up password file securely.
- Vault is not a replacement for external secret managers but helps a lot.
- Combine with roles and variables for secure automation.

REMEMBER

Encrypt = Protect
Password File = Practical
Secure Secrets = Safe Automation

ONE LINE SUMMARY

Ansible Vault helps you encrypt and protect sensitive data in your playbooks and variables, ensuring secure and reliable automation.

TAGS IN ANSIBLE

By Abhishek Singh

19. TAGS

Tags in Ansible allow you to mark tasks and execute only the required parts of a playbook. They are useful for selective execution, faster runs, and production deployments.

KEY BENEFITS

- Run only specific tasks
- Save time & resources
- Useful for large playbooks
- Ideal for production deployments
- Better control & flexibility

TOPICS COVERED

- Task Tags
- Selective Execution
- Production Deployments
- Best Practices
- Commands & Examples
- Interview Questions

HOW TAGS WORK

Add Tags to Tasks → Run Playbook with --tags → Only Tagged Tasks Execute → Faster Execution & Control → Successful Deployment

1. TASK TAGS (EXAMPLE)

Add tags to tasks in your playbook.

```
- name: Install Nginx
  yum:
    name: nginx
    state: present
  tags: [install, nginx]

- name: Start Nginx
  service:
    name: nginx
    state: started
  tags: [start, nginx]

- name: Copy Config File
  copy:
    src: nginx.conf
    dest: /etc/nginx/nginx.conf
  tags: [config, nginx]
```

2. SELECTIVE EXECUTION

Run only tasks with specific tags.

Command	Description
--tags "tag1"	Run only tasks with tag1
--tags "tag1,tag2"	Run tasks with tag1 or tag2
--skip-tags "tag1"	Skip tasks with tag1
--tags "all"	Run all tagged tasks
--skip-tags "all"	Skip all tagged tasks (Not recommended)

Examples:

```
ansible-playbook site.yml --tags "install"
ansible-playbook site.yml --tags "install,config"
ansible-playbook site.yml --skip-tags "config"
ansible-playbook site.yml --tags "all"
```

3. PRODUCTION DEPLOYMENTS

Use tags to run only required tasks in production.

Example Use Cases:

- Run only config changes
- Restart services only
- Install packages only
- Execute database migrations only

4. LIST TAGS IN PLAYBOOK

List all available tags in a playbook.

```
ansible-playbook site.yml --list-tags
```

Example Output:

```
playbook: site.yml
  play #1 (all): all        TAGS: []
    TASK TAGS: [config, install, nginx, start]
```

5. COMMON TAGS EXAMPLE

Example of practical tag usage.

```
- name: Install Packages
  yum: name={{ item }} state=present
  loop: [nginx, git, curl]
  tags: [install]

- name: Deploy Application
  copy: src=app/ dest=/var/www/app/
  tags: [deploy]

- name: Restart Service
  service: name=nginx state=restarted
  tags: [restart]
```

6. TAGS VS NO TAGS

With Tags	Without Tags
Selective Execution	Full Playbook Run
Faster Deployment	Slower Execution
Better Control	Less Control
Ideal for Production	Risk of Unwanted Changes
Efficient Resource Use	Consumes More Resources

7. BEST PRACTICES

- Use meaningful tag names
- Group related tasks with same tag
- Avoid too many tags
- Use --list-tags to explore tags
- Combine tags for better control

INTERVIEW QUESTIONS

Q1. Why use Tags in Ansible?
Ans: Tags help in running specific parts of a playbook instead of the entire playbook. This saves time, reduces risk, and is ideal for production environments.

Q2. How do you run a playbook with specific tags?
Ans: Use the --tags option followed by the tag name(s).
Example: ansible-playbook site.yml --tags "install"

Q3. How do you skip specific tags?
Ans: Use the --skip-tags option.
Example: ansible-playbook site.yml --skip-tags "config"

Q4. Can a task have multiple tags?
Ans: Yes, you can assign multiple tags to a task using a list. Example: tags: [install, nginx, critical]

IMPORTANT NOTES

- Tags are optional but highly recommended for large playbooks.
- If a task has no tag, it runs only when you use --tags "all".
- Tags do not affect task order.
- Use tags for modular and safe deployments.
- Always test tag execution before running in production.

REMEMBER

Tags = Control
Control = Safety
Safety = Successful Deployments

Run Smart.
Deploy Safer. Save Time.

ONE LINE SUMMARY

Tags in Ansible allow selective execution of tasks, giving you better control, faster deployments, and safer automation in production.

ANSIBLE

ERROR HANDLING IN ANSIBLE

By Abhishek Singh

20. ERROR HANDLING

Error handling in Ansible ensures that playbooks are more robust and can handle failures gracefully without stopping execution unexpectedly.

KEY BENEFITS

- Improves playbook reliability
- Prevents unexpected stops
- Handles failures gracefully
- Provides controlled execution flow
- Useful for production environments

TOPICS COVERED

ignore_errors	failed_when	changed_when	block	rescue	always	Interview Questions
Ignore task failures and continue execution	Define custom conditions for task failure	Define custom conditions for changed status	Group tasks together for error handling	Handle failures in a block	Tasks that always run (success or failure)	

HOW ERROR HANDLING WORKS

Execute Task → Task Fails → Apply Error Handling → Continue / Recover Execution → Playbook Completes Gracefully

1. IGNORE_ERRORS

Ignore failures and continue execution.

```yaml
- name: Install package
  yum:
    name: nginx
    state: present
  ignore_errors: yes
```

Use when a failure is non-critical.

2. FAILED_WHEN

Define custom condition for task failure.

```yaml
- name: Check disk space
  shell: df -h  | awk 'NR==2 {print $5}'
  register: disk_out
  failed_when: disk_out.stdout | int > 90
```

Task fails only if condition is true.

3. CHANGED_WHEN

Define custom condition for changed status.

```yaml
- name: Run command
  shell: /usr/bin/uptime
  register: out
  changed_when: "'load average' in out.stdout"
```

Marks task as changed only if condition is true.

4. BLOCK, RESCUE, ALWAYS

Handle failures using block, rescue and always.

```yaml
- block:
  - name: Install package
    yum:
      name: nginx
      state: present
  - name: Start service
    service:
      name: nginx
      state: started
rescue:
  - name: Handle failure
    debug:
      msg: "Something went wrong!"
always:
  - name: Cleanup / Always run
    debug:
      msg: "This runs always!"
```

- block : Tasks to try
- rescue : Runs if any task in block fails
- always : Runs always (success or failure)

Ensures controlled and predictable execution flow.

5. RESCUE EXAMPLE

Run alternative tasks on failure.

```yaml
- block:
  - name: Copy file
    copy:
      src: /tmp/src.txt
      dest: /etc/dest.txt
rescue:
  - name: Copy from backup
    copy:
      src: /backup/src.txt
      dest: /etc/dest.txt
```

Rescue block runs only if block fails.

6. ALWAYS EXAMPLE

Tasks in always block run no matter what.

```yaml
- block:
    - name: Risky Task
      command: /bin/false
  rescue:
    - debug:
        msg: "Handled failure"
  always:
    - debug:
        msg: "Always executed"
```

Useful for cleanup, notifications, logging etc.

COMPARISON SUMMARY

Feature	Purpose	Runs When
ignore_errors	Ignore task failure	On task failure
failed_when	Custom failure condition	Condition is true
changed_when	Custom changed condition	Condition is true
block	Group tasks	Always
rescue	Handle failure	When block fails
always	Always run tasks	Always

COMMON USE CASES

- Continue execution even if a task fails
- Validate conditions before failing
- Prevent false-positive changes
- Rollback or alternative actions on failure
- Always run cleanup or notifications

BEST PRACTICES

- Use ignore_errors sparingly
- Define clear failure conditions with failed_when
- Use blocks for grouped error handling
- Always provide meaningful messages
- Log and notify in always block

INTERVIEW QUESTIONS

Q1. What is Ansible error handling?
Ans: Error handling allows playbooks to manage failures gracefully without stopping execution.

Q2. How do you handle task failures without stopping the playbook?
Ans: Use ignore_errors: yes or handle using block/rescue.

Q3. What is the use of failed_when?
Ans: It defines custom conditions for task failure.

Q4. What is the use of always in a block?
Ans: Tasks in always run no matter if the block succeeds or fails.

IMPORTANT NOTES

- By default, Ansible stops on first failure.
- Use error handling to make playbooks resilient.
- Always test failure scenarios.
- Good error handling = Reliable automation.

REMEMBER

Handle Failures Smartly.
Recover Gracefully.
Automate Reliably.

Resilient Playbooks = Successful Deployments!

ONE LINE SUMMARY

Ansible error handling helps you manage failures gracefully using ignore_errors, failed_when, changed_when, block, rescue and always for reliable automation.

ANSIBLE

ANSIBLE WITH aws

By Abhishek Singh

21. ANSIBLE WITH AWS

Ansible makes it easy to automate AWS cloud infrastructure. You can provision, configure and manage AWS resources using Ansible modules.

KEY BENEFITS

- Infrastructure as Code
- Repeatable & Consistent
- Faster Provisioning
- Reduced Manual Errors
- Easy Integration with CI/CD
- Cost Effective Automation

TOPICS COVERED

EC2 Provisioning	S3 Automation	IAM Automation	Security Groups	VPC Management	Interview Questions
Launch and manage EC2 instances	Automate S3 bucket creation and management	Create users, groups and manage permissions	Manage inbound and outbound rules	Create and manage VPCs, subnets, route tables	Common AWS + Ansible interview questions

HOW ANSIBLE WORKS WITH AWS

Configure AWS Credentials (Access Key, Secret Key) → Write Ansible Playbook → Run Ansible Playbook → Ansible Modules communicate with AWS API → AWS Resources Created / Managed Successfully

COMMON AWS ANSIBLE MODULES

Resource	Ansible Module
EC2	amazon.aws.ec2_instance
S3	amazon.aws.s3_bucket
IAM User	amazon.aws.iam_user
IAM Group	amazon.aws.iam_group
Security Group	amazon.aws.ec2_security_group
VPC	amazon.aws.ec2_vpc
Subnet	amazon.aws.ec2_vpc_subnet
Route Table	amazon.aws.ec2_vpc_route_table
ELB / ALB	amazon.aws.elb_application_lb

EXAMPLE: EC2 CREATION PLAYBOOK

```yaml
---
- name: Launch EC2 Instance
  hosts: localhost
  gather_facts: no
  vars:
    region: us-east-1
    key_name: my-keypair
    instance_type: t2.micro
    image_id: ami-0c55b159cbfafe1f0  # Amazon Linux 2 AMI
    sg_id: sg-0abc12345def67890
    count: 1
  tasks:
  - name: Create EC2 Instance
    amazon.aws.ec2_instance:
      name: my-ec2-instance
      key_name: "{{ key_name }}"
      instance_type: "{{ instance_type }}"
      image_id: "{{ image_id }}"
      vpc_subnet_id: "subnet-0abc12345def67890"
      security_group: "{{ sg_id }}"
      region: "{{ region }}"
      count: "{{ count }}"
      wait: yes
      state: present
    register: ec2

  - name: Print Instance Info
    debug:
      var: ec2.instances
```

Make sure AWS credentials are configured using:

1. Environment Variables
   or
2. aws configure
   or
3. Ansible Vault

AWS CREDENTIALS SETUP

1. Environment Variables

```
export AWS_ACCESS_KEY_ID=AKIA.....
export AWS_SECRET_ACCESS_KEY=abcd.....
```

2. AWS CLI Configure

```
aws configure
```

3. Ansible Vault (Recommended)

```
ansible-vault create aws_creds.yml
```

S3 BUCKET CREATION EXAMPLE

```yaml
- name: Create S3 Bucket
  amazon.aws.s3_bucket:
    name: my-ansible-bucket-123
    region: us-east-1
    state: present
    versioning: yes
    tags:
      Environment: Dev
      Owner: Ansible
```

SECURITY GROUP EXAMPLE

```yaml
- name: Create Security Group
  amazon.aws.ec2_security_group:
    name: ansible-sg
    description: "Allow SSH and HTTP"
    vpc_id: vpc-0abc12345def67890
    region: us-east-1
    rules:
      - proto: tcp
        from_port: 22
        to_port: 22
        cidr_ip: 0.0.0.0/0
      - proto: tcp
        from_port: 80
        to_port: 80
        cidr_ip: 0.0.0.0/0
    rules_egress:
      - proto: -1
        cidr_ip: 0.0.0.0/0
    state: present
```

VPC CREATION EXAMPLE

```yaml
- name: Create VPC
  amazon.aws.ec2_vpc:
    cidr_block: 10.0.0.0/16
    region: us-east-1
    resource_tags:
      Name: my-vpc
      Environment: Dev
    state: present
```

BEST PRACTICES

- Use IAM roles with least privilege.
- Store credentials securely (Vault / IAM Role).
- Use variables and groups for reusability.
- Validate resources before creation.
- Use tags for all resources.

INTERVIEW QUESTIONS

Q1. Have you automated AWS using Ansible?
Ans: Yes, I have automated AWS using Ansible to provision and manage EC2, S3, IAM users, Security Groups, VPCs and more.

Q2. How do you create an EC2 instance using Ansible?
Ans: Using amazon.aws.ec2_instance module with required parameters like image_id, instance_type, key_name, security_group, etc.

Q3. How do you manage AWS credentials in Ansible?
Ans: Using environment variables, aws configure or Ansible Vault.

IMPORTANT NOTES

- Ansible uses boto3 under the hood to interact with AWS API.
- Always use IAM roles or temporary credentials.
- Idempotent playbooks ensure safe re-runs.
- Use check_mode for dry runs.

REMEMBER

Automate Today.
Save Time Tomorrow.

Ansible + AWS = Powerful Automation!

ONE LINE SUMMARY

Ansible with AWS helps you automate cloud infrastructure provisioning and management in a repeatable, reliable and secure way.

ANSIBLE WITH docker

By Abhishek Singh

22. ANSIBLE WITH DOCKER

Ansible can automate Docker installation, manage images, run containers and handle Docker resources efficiently across hosts.

KEY BENEFITS

- Automate Docker lifecycle
- Consistent container deployments
- Save time & reduce manual work
- Easy integration with CI/CD
- Scalable & repeatable automation

TOPICS COVERED

Docker Installation	Container Deployment	Image Pulling	Docker Management	Interview Questions
Install Docker on target hosts	Run and manage containers with ease	Pull images from Docker Hub or private registry	Manage containers, images, volumes, networks	Common Ansible + Docker interview questions

HOW ANSIBLE WORKS WITH DOCKER

Define Docker Requirements in Ansible Playbook → Ansible Connects to Target Hosts → Ansible Executes Docker Modules → Docker Performs Actions (Run, Pull, Manage) → Containers Running & Managed Successfully

COMMON ANSIBLE DOCKER MODULES

Module	Purpose
docker_host	Manage Docker daemon connection
docker_image	Pull, build or remove Docker images
docker_container	Create, start, stop, remove containers
docker_network	Manage Docker networks
docker_volume	Manage Docker volumes
docker_login	Login to Docker registry
docker_compose	Run Docker Compose applications

EXAMPLE: DEPLOY NGINX CONTAINER

```yaml
- name: Deploy Nginx Container
  hosts: all
  become: yes
  tasks:
    - name: Pull nginx image
      docker_image:
        name: nginx:latest
        source: pull

    - name: Run nginx container
      docker_container:
        name: nginx_container
        image: nginx:latest
        state: started
        restart_policy: always
        ports:
          - "80:80"
        networks:
          - name: nginx_net
```

Notes:

1. Make sure Docker is installed on the target host.
2. Use become: yes for privilege escalation.
3. Ensure required ports are open.

DOCKER INSTALLATION PLAYBOOK

```yaml
- name: Install Docker
  hosts: all
  become: yes
  tasks:
    - name: Install required packages
      package:
        name:
          - ca-certificates
          - curl
          - gnupg
          - lsb-release
        state: present

    - name: Add Docker GPG key
      apt_key:
        url: https://download.docker.com/linux/ubuntu/gpg
        state: present

    - name: Add Docker repository
      apt_repository:
        repo: "deb [arch=amd64] https://download.docker.com/linux/ubuntu \\
              {{ ansible_lsb.codename }} stable"
        state: present
        filename: docker

    - name: Install Docker
      package:
        name: docker-ce
        state: present

    - name: Ensure Docker service is running
      service:
        name: docker
        state: started
        enabled: yes
```

IMAGE PULLING EXAMPLE

```yaml
- name: Pull Redis Image
  hosts: all
  become: yes
  tasks:
    - name: Pull redis image
      docker_image:
        name: redis:7-alpine
        source: pull
```

DOCKER MANAGEMENT EXAMPLES

List Running Containers

```yaml
- name: List containers
  docker_container_info:
  register: contaniers
- debug:
    var: containers
```

Stop a Container

```yaml
- name: Stop container
  docker_container:
    name: nginx_container
    state: stopped
```

Remove a Container

```yaml
- name: Remove container
  docker_container:
    name: nginx_container
    state: absent
```

BEST PRACTICES

- Use docker_image to ensure desired image is present.
- Always set restart_policy for critical containers.
- Use variables for image names, ports, and other parameters.
- Keep playbooks idempotent and test in lower environments.
- Use docker_compose for multi-container applications.

INTERVIEW QUESTIONS

Q1. How do you manage Docker using Ansible?
Ans: By using Ansible Docker modules like docker_image, docker_container, docker_network, etc.

Q2. How do you pull a Docker image using Ansible?
Ans: Using the docker_image module with source: pull.

Q3. How do you run a container using Ansible?
Ans: Using the docker_container module with required parameters like image, name, ports, etc.

Q4. How do you remove a container using Ansible?
Ans: Set state: absent in docker_container module.

IMPORTANT NOTES

- Docker must be installed before managing containers.
- Ansible communicates with Docker daemon via API.
- Use privilege escalation (become: yes) where required.
- Store sensitive data like registry credentials in Ansible Vault.

REMEMBER

Automate Docker.
Deploy Faster.
Manage Better.
Scale Smarter!

ONE LINE SUMMARY

Ansible with Docker helps you automate installation, deployment and management of containers at scale with consistency and ease.

ANSIBLE WITH kubernetes

By Abhishek Singh

23. ANSIBLE WITH KUBERNETES

Ansible makes it easy to automate Kubernetes operations and deployments using the powerful kubernetes modules.

KEY BENEFITS

- Automate K8s deployments consistently
- Manage K8s resources as Code
- Reduce manual errors
- Easy integration with CI/CD
- Idempotent & declarative
- Faster application delivery

TOPICS COVERED

Deployments	Services	ConfigMaps	Secrets	Helm Integration	Interview Questions
Deploy and update applications on Kubernetes	Expose applications within or outside the cluster	Manage non-sensitive configuration data separately	Manage sensitive information securely	Install and manage Helm charts using Ansible	Common Ansible + Kubernetes interview questions

HOW ANSIBLE WORKS WITH KUBERNETES

Define K8s Resources in Ansible Playbook → Ansible Connects to Kubernetes API → Ansible Executes K8s Modules → Kubernetes API Applies Changes → Application Deployed & Running

COMMON ANSIBLE KUBERNETES MODULES

Module	Purpose
kubernetes.core.k8s	General purpose resource management
kubernetes.core.k8s_info	Get information about resources
kubernetes.core.k8s_scale	Scale deployments, statefulsets
kubernetes.core.k8s_exec	Execute commands in pods
kubernetes.core.k8s_log	Fetch logs from pods
community.kubernetes.helm	Manage Helm charts
kubernetes.core.k8s_config	Manage kubeconfig settings

KUBERNETES RESOURCES EXAMPLES

Deployment

```
definition:
  apiVersion: apps/v1
  kind: Deployment
  metadata:
    name: my-app
  spec:
    replicas: 2
```

Service

```
definition:
  apiVersion: v1
  kind: Service
  metadata:
    name: my-service
  spec:
    type: ClusterIP
    ports:
    - port: 80
      targetPort: 80
```

ConfigMap

```
definition:
  apiVersion: v1
  kind: ConfigMap
  metadata:
    name: app-config
  data:
    key: value
```

Secret

```
definition:
  apiVersion: v1
  kind: Secret
  metadata:
    name: app-secret
  type: Opaque
  stringData:
    password: mypass
```

Helm Release

```
- name: Install Nginx Helm Chart
  community.kubernetes.helm:
    name: nginx
    chart_ref: nginx
    release_namespace: default
    create_namespace: yes
```

EXAMPLE: DEPLOY NGINX APPLICATION

```
- name: Deploy Nginx to Kubernetes
  hosts: localhost
  gather_facts: no
  vars:
  - namespace: default
  tasks:
   - name: Create Deployment
     kubernetes.core.k8s:
       state: present
       definition:
         apiVersion: apps/v1
         kind: Deployment
         metadata:
           name: nginx-deployment
           namespace: "{{ namespace }}"
         spec:
           replicas: 3
           selector:
             matchLabels:
               app: nginx
           template:
             metadata:
               labels:
                 app: nginx
             spec:
               containers:
                - name: nginx
                  image: nginx:latest
                  ports:
                   - containerPort: 80
```

Notes:

1. Make sure kubeconfig is available on the control machine.
2. Install the required collection:

```
ansible-galaxy collection install kubernetes.core
```

3. The k8s module is idempotent.

USEFUL KUBERNETES MODULE ACTIONS

Get Resources Info

```
- name: Get Pods
  kubernetes.core.k8s_info:
  kind: Pod
  namespace: default

  register: pods
```

Scale Deployment

```
- name: Scale Deployment
  kubernetes.core.k8s_scale:
  state: present
  kind: Deployment
  name: nginx-deployment
  namespace: default
  replicas: 5
```

Execute Command in Pod

```
- name: Exec command
  kubernetes.core.k8s_exec:
  namespace: default
  pod: nginx-abc123
  command: /bin/sh
  stdin: false
  tty: false
```

Fetch Logs

```
- name: Get Pod Logs
  kubernetes.core.k8s_log:
  namespace: default
  pod: nginx-abc123
  container: nginx
```

BEST PRACTICES

- Use fully qualified collection name: kubernetes.core.k8s
- Store sensitive data in Kubernetes Secrets
- Use namespaces to isolate environments
- Use vars and templates for reusability
- Validate playbooks with --check mode
- Integrate with CI/CD for GitOps workflows

INTERVIEW QUESTIONS

Q1. How do you deploy applications to Kubernetes using Ansible?
Ans: By using kubernetes.core.k8s module to define and apply resources like Deployments, Services, ConfigMaps, etc.

Q2. How do you store secrets in Kubernetes using Ansible?
Ans: Use kubernetes.core.k8s to create Secret resources or integrate with external secret managers.

Q3. How do you update a Kubernetes Deployment using Ansible?
Ans: Update the image or other fields in the Deployment definition and re-apply using k8s module.

Q4. How do you integrate Helm with Ansible?
Ans: Use community.kubernetes.helm module to install, upgrade or uninstall Helm charts.

IMPORTANT NOTES

- Ansible communicates with Kubernetes API using kubeconfig.
- Idempotent operations ensure safe re-runs.
- Use check_mode for dry-run and preview changes.
- Always test in lower environments before production.

REMEMBER

Ansible + Kubernetes = Powerful Automation for Modern Cloud Native Applications!

ONE LINE SUMMARY

Ansible helps you automate Kubernetes deployments and operations reliably, consistently and at scale.

CI/CD INTEGRATION

By Abhishek Singh

24. CI/CD INTEGRATION

Ansible fits perfectly into CI/CD pipelines to automate infrastructure provisioning, configuration, testing and deployment with consistency and speed.

KEY BENEFITS

- Automate infrastructure as code
- Consistent & repeatable deployments
- Reduce manual errors
- Faster releases & feedback
- Easy integration with CI/CD tools
- Better collaboration & visibility

TOPICS COVERED

Jenkins Integration
Trigger Ansible playbooks using Jenkins pipelines for automated builds and deployments.

GitHub Actions Integration
Use GitHub Actions to run Ansible playbooks on push, pull request or schedule.

GitLab Integration
Leverage GitLab CI/CD to execute Ansible playbooks in pipeline stages.

Azure DevOps Integration
Integrate Ansible with Azure DevOps pipelines for continuous delivery and release.

Interview Questions
Common CI/CD + Ansible interview questions.

HOW ANSIBLE FITS INTO CI/CD PIPELINE

Code Commit → Build & Test → Run Ansible Playbook → Deploy / Configure Infrastructure → Release → Monitor & Feedback

1. JENKINS INTEGRATION

Jenkinsfile (Example)

```
pipeline {
  agent any
  stages {
    stage('Checkout') {
      steps { git 'https://github.com/org/repo.git' }
    }
    stage('Run Ansible') {
      steps {
        sh 'ansible-playbook -i inventory
        site.yml -e env=prod'
      }
    }
  }
}
```

Notes:
- Install Ansible on Jenkins agent
- Use credentials for SSH access

2. GITHUB ACTIONS INTEGRATION

.github/workflows/ansible.yml

```
name: Ansible Playbook
on:
  push:
    branches: [ main ]
jobs:
  deploy:
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v4
      - name: Install Ansible
        run: |
          sudo apt-get update
          sudo apt-get install -y ansible
      - name: Run Ansible Playbook
        run: ansible-playbook -i inventory
        site.yml -e env=prod
```

Notes:
- Store SSH key in GitHub Secrets
- Use secrets for vault password

3. GITLAB INTEGRATION

.gitlab-ci.yml

```
stages:
  - deploy

ansible_deploy:
  stage: deploy
  image: alpine:latest
  before_script:
    - apk add --no-cache ansible openssh
  script:
    - ansible-playbook -i inventory
      site.yml -e env=prod
  only:
    - main
```

Notes:
- Configure CI/CD variables in GitLab
- Protect sensitive variables

4. AZURE DEVOPS INTEGRATION

azure-pipelines.yml

```
trigger:
- main

pool:
  vmImage: 'ubuntu-latest'

steps:
- checkout: self
- task: UsePythonVersion@0
  inputs:
    versionSpec: '3.x'
- script: |
    pip install ansible
  displayName: 'Install Ansible'
- script: |
    ansible-playbook -i inventory
    site.yml -e env=prod
  displayName: 'Run Ansible Playbook'
```

Notes:
- Use Azure Key Vault for secrets
- Use Service Connections if needed

BEST PRACTICES

- Keep playbooks idempotent and environment agnostic.
- Store inventory and group_vars in version control.
- Use Ansible Vault for secrets and sensitive data.
- Fail fast and notify on errors.
- Use tags and extra-vars for flexible deployments.
- Validate playbooks with --check before production.

COMMON CI/CD USE CASES

Infrastructure Provisioning
Provision cloud resources and configure using Ansible.

Application Deployment
Deploy and configure applications to servers.

Configuration Management
Ensure systems remain in desired state.

Rollback & Recovery
Quickly rollback to previous stable state.

INTERVIEW QUESTIONS

Q1. How does Ansible fit into CI/CD?
Ans: Ansible automates provisioning, configuration and deployment in CI/CD pipelines ensuring consistent, repeatable and reliable releases.

Q2. Which CI/CD tools have you integrated with Ansible?
Ans: Jenkins, GitHub Actions, GitLab CI/CD, Azure DevOps.

Q3. How do you handle secrets in CI/CD with Ansible?
Ans: Use Ansible Vault and store credentials in CI/CD secrets management (Jenkins Credentials, GitHub Secrets, etc.).

Q4. How do you test Ansible playbooks in CI/CD?
Ans: Use ansible-playbook --check and linters like ansible-lint.

IMPORTANT NOTES

- Ansible works in agentless mode, easy to integrate.
- Keep inventories and playbooks in the repository.
- Use environment specific variables.
- Leverage artifacts for logs and reports.
- Always test in lower environments before production deployment.

REMEMBER

Ansible + CI/CD = Faster Delivery, Consistent Deployments, Happier Teams!

ONE LINE SUMMARY

Ansible in CI/CD automates infrastructure and application delivery, ensuring consistency, speed and reliability across environments.

REAL-TIME PROJECT DESIGN

END-TO-END ANSIBLE PROJECT

By Abhishek Singh

25. REAL-TIME AROJECT DESIGN

This end-to-end project shows how Ansible integrates with modern DevOps tools to deliver a fully automated CI/CD pipeline and deploy applications to Kubernetes.

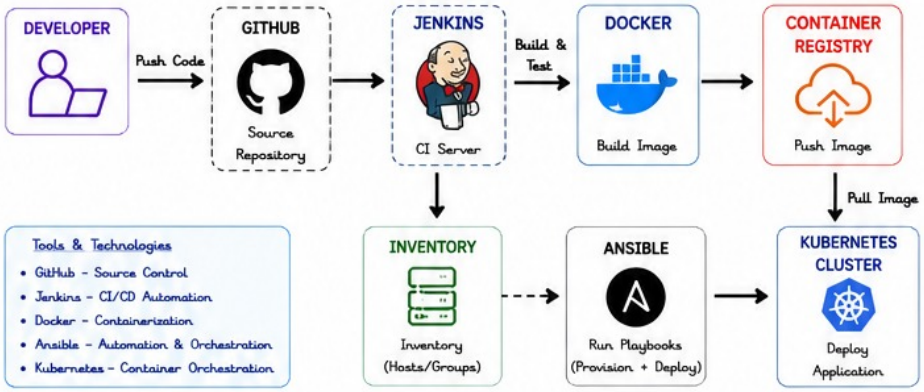
KEY HIGHLIGHTS

- End-to-end automation
- CI/CD pipeline integration
- Infrastructure as Code with Ansible
- Containerized application delivery
- Kubernetes deployment
- Production-ready architecture

END-TO-END PIPELINE FLOW

GitHub	Jenkins	Build	Docker	Push Image	Ansible	Kubernetes Deployment
Code commit triggers the pipeline	Pipeline triggered and Ansible playbooks executed	Code is built and tested successfully	Docker image is created from build	Image is pushed to container registry	Ansible deploys and configures infrastructure	Application is deployed to Kubernetes cluster

PROJECT ARCHITECTURE DIAGRAM



ANSIBLE ROLE STRUCTURE (EXAMPLE)

```
roles/
  ├── common/
  ├── docker/
  ├── kubernetes/
  ├── app-deploy/
  └── requirements.yml
playbooks/
  ├── site.yml
  ├── docker.yml
  ├── k8s-deploy.yml
inventory/
  ├── hosts.ini
  └── group_vars/
      ├── all.yml
```

Role Responsibilities

- common : Base packages, users, security
- docker : Install & configure Docker
- kubernetes : Install kubeadm, kubectl, configure cluster
- app-deploy : Deploy application manifests and configure services

ANSIBLE PLAYBOOK FLOW (HIGH LEVEL)

1. **Provision Infra** — Prepare servers, install dependencies
2. **Setup Docker** — Install & start Docker, configure daemon
3. **Setup Kubernetes** — Install & configure Kubernetes cluster
4. **Deploy Application** — Apply Kubernetes manifests, configmaps, services
5. **Validate Deployment** — Check pods, services and application health

Delivered Outcome

- Automated CI/CD Pipeline
- Containerized App Delivery
- Kubernetes Orchestration
- Scalable & Reliable Deployment
- Reproducible Infrastructure

SAMPLE ANSIBLE PLAYBOOK (site.yml)

```yaml
---
- name: End-to-End Deployment
  hosts: all
  become: yes
  roles:
    - common
    - docker
    - kubernetes
    - app-deploy
```

SAMPLE KUBERNETES DEPLOYMENT (deployment.yml)

```yaml
apiVersion: apps/v1
kind: Deployment
metadata:
  name: my-app
spec:
  replicas: 3
  selector:
    matchLabels:
      app: my-app
  template:
    metadata:
      labels:
        app: my-app
    spec:
      containers:
        - name: my-app
          image: myrepo/my-app:latest
          ports:
          - - containerPort: 80
```

BEST PRACTICES

- Use version control for all playbooks
- Keep playbooks idempotent & modular
- Store secrets securely with Ansible Vault
- Use dynamic inventory for scalability
- Validate at each stage of pipeline
- Monitor deployments and enable rollback

INTERVIEW QUESTIONS

Q1. Explain your Ansible project architecture.
Ans: The project follows a CI/CD pipeline where code is committed to GitHub, Jenkins triggers the build, Docker creates the image, and Ansible deploys the application on Kubernetes.

Q2. How does Ansible fit into CI/CD?
Ans: Ansible automates infrastructure provisioning and application deployment, ensuring consistency and reliability.

Q3. Why did you use roles in your Ansible project?
Ans: Roles help in modularity, reusability and maintainability of playbooks.

Q4. How do you handle secrets in your project?
Ans: Using Ansible Vault to encrypt and manage sensitive data like passwords and tokens.

IMPORTANT NOTES

- Keep environments (dev, staging, prod) separate.
- Use variables and templates for reusability.
- Always test in lower environments before production.
- Implement rollback strategy for safe deployments.
- Document your playbooks and pipeline clearly.

REMEMBER

Automate Everything,
Deploy Anywhere,
Scale Effortlessly!

ONE LINE SUMMARY

Ansible brings automation, consistency and reliability to CI/CD pipelines, enabling seamless delivery of applications to Kubernetes.

PRODUCTION TROUBLESHOOTING

By Abhishek Singh

26. PRODUCTION TROUBLESHOOTING

Troubleshooting is a key skill for any Ansible Engineer. Quickly identify issues, analyze logs and fix problems to keep automation smooth in production.

TROUBLESHOOTING PRINCIPLES

- Understand the Error Clearly
- Reproduce the Issue
- Check Logs & Verbose Output
- Isolate the Root Cause
- Apply the Fix and Re-run
- Prevent & Improve

COMMON SCENARIOS & SOLUTIONS

1 Playbook Failed Midway

- Check the error message
- Run with -vvv for details
- Check failed task & logs
- Fix and re-run from failed task or use --start-at-task

2 SSH Access Denied

- Verify SSH key setup
- Check user permissions
- Validate known_hosts
- Test SSH manually

3 Inventory Mismatch

- Verify hosts in inventory
- Check group and host names
- Validate dynamic inventory script/output
- Run ansible-inventory --list

4 Variable Not Loading

- Check variable file path
- Verify variable precedence
- Use debug to print variables
- Ensure file format (YAML) is correct

5 Service Not Restarting

- Check service status
- Verify service name
- Look for errors in logs
- Ensure handler is triggered

6 Vault Password Failure

- Verify vault password
- Check vault ID if using multiple vaults
- Ensure vault file is encrypted correctly

7 Dynamic Inventory Issue

- Check script/Plugin logs
- Verify cloud credentials
- Validate inventory output
- Test script manually

QUICK COMMANDS

- ansible all -m ping
- ansible-playbook site.yml -vvv
- ansible-inventory --list
- ansible-config dump
- ansible-doc <module>
- journalctl -u <service> -xe

TROUBLESHOOTING WORKFLOW

1. **Identify Issue** — Observe error and symptoms
2. **Gather Info** — Check logs, output and configuration
3. **Analyze Root Cause** — Understand what went wrong
4. **Fix the Issue** — Update playbook, config or environment
5. **Verify & Re-run** — Re-run playbook and validate success
6. **Prevent** — Document fix and improve automation

DEBUGGING TIPS

```
# Run with more verbosity
ansible-playbook site.yml -vvv
# Limit to a specific host
ansible-playbook site.yml -l <host> -vvv
# Start from a specific task
ansible-playbook site.yml --start-at-task="Task Name"
# Check variable values
- debug: var=my_variable
# Check facts
ansible <host> -m setup
# Check service status
- shell: systemctl status <service>
  register: svc_status
- debug: var=svc_status.stdout_lines
```

BEST PRACTICES

- Use -vvv for detailed output.
- Validate inventory and variables before running.
- Use blocks with rescue to handle failures.
- Test in lower environments first.
- Use Ansible Lint to catch issues early.
- Keep playbooks idempotent and modular.
- Monitor logs and system status regularly.

INTERVIEW QUESTIONS

Q1. How do you troubleshoot Ansible failures?
Ans: I check the error message, run with -vvv, analyze logs, validate inventory, variables and environment, fix the issue and re-run.

Q2. What steps do you follow when a playbook fails?
Ans: Identify the failure, gather logs, reproduce, fix the root cause and verify.

Q3. How do you handle SSH access issues?
Ans: I verify SSH keys, user permissions, known_hosts and test SSH manually.

Q4. What if variables are not loading?
Ans: I check variable file path, precedence and use debug to verify values.

IMPORTANT NOTES

- Always check the error message carefully.
- Verbose mode is your best friend.
- Logs contain the root cause.
- Small changes, test and validate.
- Document issues and solutions for future reference.

REMEMBER

Good troubleshooting saves time, reduces downtime and builds reliable automation!

ONE LINE SUMMARY

Troubleshoot smartly, fix quickly and automate reliably.

ADVANCED CONCEPTS

By Abhishek Singh

27. ADVANCED CONCEPTS

Advanced Ansible features help you write efficient, scalable and production-ready playbooks for complex environments.

KEY BENEFITS

- Improve performance & efficiency
- Handle complex environments
- Reduce execution time
- More control over task execution
- Enable zero-downtime deployments
- Better scalability and reliability

TOPICS COVERED

Async Tasks	Polling	Delegation	Local Actions	Serial Deployment	Rolling Updates	Parallel Execution
Run long running tasks in the background.	Check the status of async tasks periodically.	Delegate tasks to another host.	Run tasks on the control node instead of remote.	Deploy to a limited number of hosts at a time.	Update applications with zero or minimal downtime.	Execute tasks concurrently to speed up playbooks.

1. ASYNC TASKS & POLLING

```yaml
- name: Start long running job
  command: /opt/scripts/long-task.sh
  async: 3600          # Run up to 1 hour
  poll: 0              # Fire and forget
  register: job_result

- name: Check job status
  async_status:
    jid: "{{ job_result.ansible_job_id }}"
  register: job_status
  until: job_status.finished
  retries: 30
  delay: 10
```

Notes

- async starts the task in background.
- poll: 0 means Ansible won't wait.
- Use async_status to check progress.

2. DELEGATION

```yaml
- name: Get disk usage from DB server
  shell: df -h
  delegate_to: db-server
  register: disk_out
```

Use Cases

- Run tasks on load balancer, DB, or bastion host
- Centralized logging, backups, notifications

3. LOCAL ACTIONS

```yaml
- name: Create local backup directory
  file:
    path: /tmp/backup
    state: directory
  delegate_to: localhost
  run_once: true
```

Use Cases

- Work with local files
- Generate reports
- Interact with local tools/APIs

4. SERIAL DEPLOYMENT

```yaml
- hosts: webservers
  serial: 2          # Deploy to 2 servers at a time
  tasks:
    - name: Deploy application
      copy:
        src: app.war
        dest: /opt/tomcat/webapps/

    - name: Restart service
      service:
        name: tomcat
        state: restarted
```

How it works

Batch 1 (2 hosts) → Batch 2 (2 hosts) → Batch 3 (1 host)

5. ROLLING UPDATES

```yaml
- hosts: webservers
  serial: 1
  tasks:
    - name: Stop old container
      docker_container:
        name: myapp
        state: stopped
    - name: Pull new image
      docker_image:
        name: myapp:latest
        source: pull
    - name: Start new container
      docker_container:
        name: myapp
        image: myapp:latest
        state: started
```

Rolling Update Flow

Take 1 host out of service → Deploy update & run health check → Put host back in service → Repeat for next host

6. PARALLEL EXECUTION

```yaml
- hosts: all
  strategy: free  # or linear (default)
  tasks:
    - name: Run command
      shell: uptime
```

Strategies

- linear (default): host by host
- free: tasks run independently on each host (faster)

7. OTHER USEFUL OPTIONS

Option	Description	Example
throttle	Limit concurrent tasks	throttle: 5
run_once	Run task only once	run_once: true
delegate_facts	Store facts from delegated host	delegate_facts: true
any_errors_fatal	Stop play on any host failure	any_errors_fatal: true
max_fail_percentage	Stop if % of hosts fail	max_fail_percentage: 20

SERIAL vs ROLLING DEPLOYMENT

Serial Deployment	Rolling Deployment
• Update a fixed number of hosts at a time • Waits for tasks to complete before next batch • Useful for stateful applications	• Update one host (or small batch) at a time • Ensures zero/minimal downtime • Best for web apps and stateless services

VS

BEST PRACTICES

- Use async for long running tasks.
- Use serial for safe deployments.
- Use rolling updates for zero downtime.
- Delegate heavy tasks to appropriate hosts.
- Use proper error handling and retries.
- Test in lower environments before production.

INTERVIEW QUESTIONS

Q1. What is Serial Execution?
Ans: Serial execution allows tasks to run on a limited number of hosts at a time using the serial keyword.

Q2. How do you perform rolling deployments?
Ans: Use serial with small batch size (e.g., 1) and health checks between updates to achieve rolling deployments.

Q3. What is the difference between async and poll?
Ans: async runs the task in background, poll checks its status at regular intervals.

IMPORTANT NOTES

- Default strategy is linear.
- Use async + poll to avoid timeouts.
- run_once runs task only on first host.
- Delegate wisely to avoid bottlenecks.
- Rolling updates reduce downtime and risk.

REMEMBER

Advanced Features = Smarter Automation, Better Performance, Happier Teams!

ONE LINE SUMMARY

Leverage advanced concepts to build fast, reliable and production-ready automation with Ansible.

REAL-TIME INTERVIEW SCENARIOS

By Abhishek Singh

28. REAL-TIME INTERVIEW SCENARIOS

These real-world scenarios are typically discussed in Ansible interviews to evaluate your practical understanding and problem-solving skills.

WHAT INTERVIEWERS LOOK FOR

- Practical Ansible knowledge
- Playbook design approach
- Handling scale & failures
- Idempotency & reliability
- Security best practices
- Automation mindset

1. DEPLOY APPLICATION TO 500 SERVERS WITH ZERO DOWNTIME

Deploy the latest version of an application to 500 servers without any downtime.

Key Points
- Rolling updates
- Batch (serial) execution
- Health checks
- Rollback on failure

Ansible Features
- serial, rolling update
- max_fail_percentage
- health check with uri
- handlers & notify

2. RESTART ONLY FAILED SERVICES

Identify failed services on multiple servers and restart only those services.

Key Points
- Check service status
- Conditional restart
- Avoid unnecessary restarts
- Log audit

Ansible Features
- service_facts
- failed_when, when
- handlers
- register & condition

3. PATCH ALL LINUX SERVERS AUTOMATICALLY

Automate OS patching for all Linux servers with minimal manual intervention.

Key Points
- Update packages
- Reboot if required
- Kernel update handling
- Report status

Ansible Features
- yum/apt modules
- reboot module
- register & notify
- conditional tasks

4. ROTATE SSL CERTIFICATES ACROSS INFRASTRUCTURE

Rotate SSL certificates on web servers, load balancers, and dependent services.

Key Points
- Secure certificate distribution
- Update across services
- Minimal downtime
- Verify expiry after rotation

Ansible Features
- copy/template modules
- notify & handlers
- service reload
- openssl / command module

5. CREATE AWS INFRASTRUCTURE AND DEPLOY APPLICATION AUTOMATICALLY

Provision AWS infrastructure (EC2, VPC, SG, IAM, S3) and deploy the application end-to-end.

Key Points
- Infrastructure as Code
- Secure & repeatable setup
- Deploy after provisioning
- Clean up & teardown

Ansible Features
- amazon.aws collection
- ec2, vpc, s3, iam modules
- dynamic inventory
- ansible-playbook chaining

6. BLUE-GREEN DEPLOYMENT USING ANSIBLE

Deploy new version (Green) alongside current (Blue) and switch traffic with zero downtime.

Key Points
- Two identical environments
- Health validation
- Traffic switch
- Rollback strategy

Ansible Features
- load balancer update
- health checks
- serial/rolling strategy
- rollback on failure

7. CANARY DEPLOYMENT USING ANSIBLE

Deploy new version to a small set of servers (10%), validate and then roll out to all.

Key Points
- Small subset first
- Monitor & validate
- Gradual rollout
- Auto rollback on issues

Ansible Features
- serial, limit
- traffic control
- health checks
- max_fail_percentage

8. DISASTER RECOVERY AUTOMATION

Automate disaster recovery process to restore services and data with minimal downtime.

Key Points
- Backup verification
- Restore infrastructure
- Data recovery
- Service validation

Ansible Features
- backup & fetch modules
- restore automation
- idempotent playbooks
- alerting & reporting

HOW TO APPROACH ANY SCENARIO IN INTERVIEW

1. **Understand** — Clarify requirements, assumptions & constraints.
2. **Plan** — Design high-level approach and playbook structure.
3. **Implement** — Use Ansible best practices & modular roles.
4. **Validate** — Test in lower environment, validate results.
5. **Monitor** — Monitor execution, logs and system health.
6. **Improve** — Review, optimize and make it idempotent.

INTERVIEW QUESTIONS

Q1. How do you deploy an application with zero downtime using Ansible?
Ans: Use rolling updates with serial, health checks and rollback on failure.

Q2. How will you restart only failed services?
Ans: Check service status using service_facts and restart conditionally.

Q3. How do you automate patching on Linux servers?
Ans: Use package modules (yum/apt), reboot if required and report status.

Q4. How do you rotate SSL certificates using Ansible?
Ans: Copy new certs, reload services and verify certificate expiry.

Q5. How do you perform Blue-Green or Canary deployment?
Ans: Deploy to a subset, validate with health checks and switch traffic.

Q6. How do you handle disaster recovery with Ansible?
Ans: Automate backup, restore, infra recovery and service validation.

IMPORTANT NOTES

- Always ensure idempotency in every playbook.
- Use variables & inventory for flexibility.
- Always test in non-production before execution.
- Log everything and handle failures gracefully.
- Security, performance and reliability are key.

REMEMBER

Automation is not just about writing playbooks, it's about solving real business problems reliably and efficiently.

ONE LINE SUMMARY

Real-world scenarios test your ability to design, automate and troubleshoot complex infrastructure with Ansible.

TOP 100 ANSIBLE INTERVIEW QUESTIONS

By Abhishek Singh

• 100 Must-Know Questions to Crack Any Ansible Interview •

BEGINNER (1–25)

01. What is Ansible?
02. Why Ansible?
03. What is Inventory?
04. What are Modules?
05. What is YAML?
06. What are Playbooks?
07. What are Roles?
08. How does Ansible work?
09. What is ansible.cfg?
10. What is a Host pattern?
11. How to ping hosts in Ansible?
12. What is a Task?
13. What is a Play?
14. What is a Handler?
15. What is an Ad-hoc command?
16. How to run a Playbook?
17. What is idempotency?
18. What is check mode?
19. How to list all hosts?
20. What is a Group?
21. How to create Inventory?
22. What is become (sudo)?
23. How to use become_user?
24. What is a Module example?
25. How to copy a file?

INTERMEDIATE (26–50)

26. What is variable precedence?
27. How to define variables?
28. What are facts?
29. How to register variables?
30. What is when condition?
31. How to use loops?
32. What is with_items?
33. What is until?
34. What are Tags?
35. How to use Tags in Playbooks?
36. What are Handlers?
37. When are Handlers executed?
38. What is notify?
39. What is Meta module?
40. What is include_tasks?
41. What is import_tasks?
42. What is include_role?
43. What is import_role?
44. What is Dynamic Inventory?
45. How to create custom inventory?
46. What is Ansible Vault?
47. How to encrypt variables?
48. How to use Vault in Playbook?
49. What is Jinja2?
50. How to use templates (Jinja2)?

ADVANCED (51–75)

51. What is Ansible architecture?
52. How Ansible works internally?
53. What is the control node?
54. What is connection plugin?
55. What are strategy plugins?
56. What is async tasks?
57. How does polling work?
58. What is fire_and_forget mode?
59. How to use async and poll?
60. What is Delegation?
61. How to delegate tasks?
62. What is run_once?
63. What are Local Actions?
64. What is serial execution?
65. How serial works?
66. What is rolling deployment?
67. How to perform rolling updates?
68. How to limit batch size?
69. What is max_fail_percentage?
70. How to handle failures?
71. What is block/rescue/always?
72. How to debug in Ansible?
73. How to use Ansible in CI/CD?
74. How to integrate Jenkins?
75. How to integrate GitHub Actions?

SCENARIO-BASED (76–100)

76. How to deploy app to 500 servers?
77. How to achieve zero downtime?
78. How to restart only failed services?
79. How to patch all Linux servers?
80. How to rotate SSL certificates?
81. How to automate AWS infra?
82. How to deploy app on AWS?
83. How to automate EC2 creation?
84. How to automate S3 bucket?
85. How to deploy on Kubernetes?
86. How to update K8s deployments?
87. How to perform Blue-Green Deploy?
88. How to perform Canary Deploy?
89. How to rollback deployments?
90. How to handle config drift?
91. How to monitor with Ansible?
92. How to integrate with Docker?
93. How to build CI/CD pipeline?
94. How to automate backups?
95. How to automate DR (Disaster Recovery)?
96. How to use Ansible for Multi-Cloud?
97. How to handle Inventory mismatch?
98. How to troubleshoot Playbook failure?
99. How to secure Ansible?
100. Best practices for Ansible?

QUICK REFERENCE – KEY CONCEPTS

Core Concepts
- Inventory, Modules,
- Playbooks, Roles,
- Tasks, Handlers,
- Variables, Facts

Execution
- Ad-hoc, Playbook,
- Strategy, Serial
- Async, Polling
- Parallel Execution

Logic & Flow
- Loops, Conditions,
- Tags, Blocks,
- Handlers, Meta,
- Includes

Security
- Vault, SSH,
- Privilege Escalation
- Best Practices
- Access Control

Integrations
- AWS, Docker,
- Kubernetes, CI/CD
- Jenkins, GitHub,
- Monitoring, DR

Advanced
- Delegation, Local Actions,
- Jinja2, Dynamic Inventory
- Rolling Updates, Blue-Green
- Canary, Troubleshooting

PRO TIP

Understand concepts clearly, practice hands-on, build real projects, and solve scenario-based problems to crack any Ansible interview!

INTERVIEW TIPS

- ✓ Understand fundamentals deeply.
- ✓ Practice real-world scenarios.
- ✓ Write clean, modular playbooks.
- ✓ Explain with examples.
- ✓ Stay updated with Ansible latest features.

IMPORTANT NOTES

- Hands-on experience is most important.
- Always explain with real examples.
- Focus on problem-solving approach.
- Know Ansible best practices.
- Keep learning and stay curious.

REMEMBER

Ansible is simple, but mastering it takes practice, consistency and real-world experience!

ONE LINE SUMMARY

Master the top 100 Ansible interview questions and be ready to ace any DevOps, SRE or Automation Engineer role!

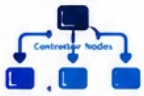
COMPLETE ANSIBLE INTERVIEW REVISION SHEET

By Abhishek Singh

★★ YOUR COMPLETE ROADMAP TO CRACK ANY ANSIBLE INTERVIEW ★★

This one-sheet revision guide covers all must-know topics, key concepts, integrations, automation, real-time scenarios and interview questions to help you crack DevOps, Cloud, SRE and Automation Engineer interviews.

MUST KNOW TOPICS

1. Ansible Architecture


Understand core architecture, components and how Ansible works.

2. Installation

Install Ansible on various OS and configure properly.

3. Inventory

Static inventory, host groups, variables in inventory.

4. Dynamic Inventory

Use dynamic inventory with cloud providers and plugins.

5. Modules

Core, custom and 3rd party modules usage.

6. Ad-Hoc Commands

Run one-liner Commands for quick automation tasks.

7. Playbooks

Write, run and organize automation with playbooks.

8. YAML


YAML syntax, structure and best practices.

9. Variables

Define, use and manage variables efficiently.

10. Facts

Gathering and using facts for dynamic automation.

11. Loops

Looping with item, with_items, loop and loop control.

12. Conditions

When, unless and conditional execution in playbooks.

13. Handlers

Notify and run handlers on specific events.

14. Templates

Use Jinja2 templates for dynamic configurations.

15. Roles

Organize playbooks using roles for reusability and scalability.

16. Ansible Galaxy

Find, install and share roles using Ansible Galaxy.

17. Vault

Encrypt data, manage secrets with Ansible Vault.

18. Tags

Run specific parts of playbook using tags.

19. Error Handling

Handle failures, ignore errors and ensure resilient playbooks.

20. AWS Automation


Automate AWS services like EC2, S3, IAM, VPC and more.

21. Docker Automation

Install, configure and manage Docker using Ansible.

22. Kubernetes Automation

Deploy and manage Kubernetes resources with Ansible.

23. Jenkins Integration

Integrate Ansible with Jenkins for CI/CD.

24. GitHub Actions Integration

Automate workflows using Ansible in GitHub Actions.

25. CI/CD Automation

Build complete CI/CD pipelines with Ansible.

26. Rolling Deployment

Perform rolling updates with zero/minimal downtime.

27. Blue-Green Deployment


Switch traffic between environments for zero downtime.

28. Canary Deployment

Deploy to small subset, validate and gradually rollout.

29. Production Troubleshooting

Debug playbooks, fix issues and optimize performance.

30. Real-Time Scenarios

Solve real-world problems in interview.

31. Top 100 Interview Questions

Practice the most important Ansible interview questions.

32. End-to-End DevOps Project Using Ansible

Design, automate and deliver real-world end-to-end project.


Master these topics and you are READY to crack any interview with CONFIDENCE!


This 30-page roadmap is sufficient for most DevOps, Cloud, SRE, Platform Engineer, and Infrastructure Automation interviews from 2 to 8+ years of experience.


DevOps Engineer


Cloud Engineer


SRE


Platform Engineer

INTERVIEW SUCCESS TIPS
- Understand concepts deeply
- Practice real-world scenarios
- Build end-to-end projects
- Stay updated with latest features
- Communicate clearly in interviews

> Automation is not just about writing playbooks, it's about solving real business problems reliably and efficiently.

KEEP PRACTICING, KEEP AUTOMATING, KEEP GROWING!

★ AUTOMATE EVERYTHING. DEPLOY ANYWHERE. SCALE EFFORTLESSLY. ★